



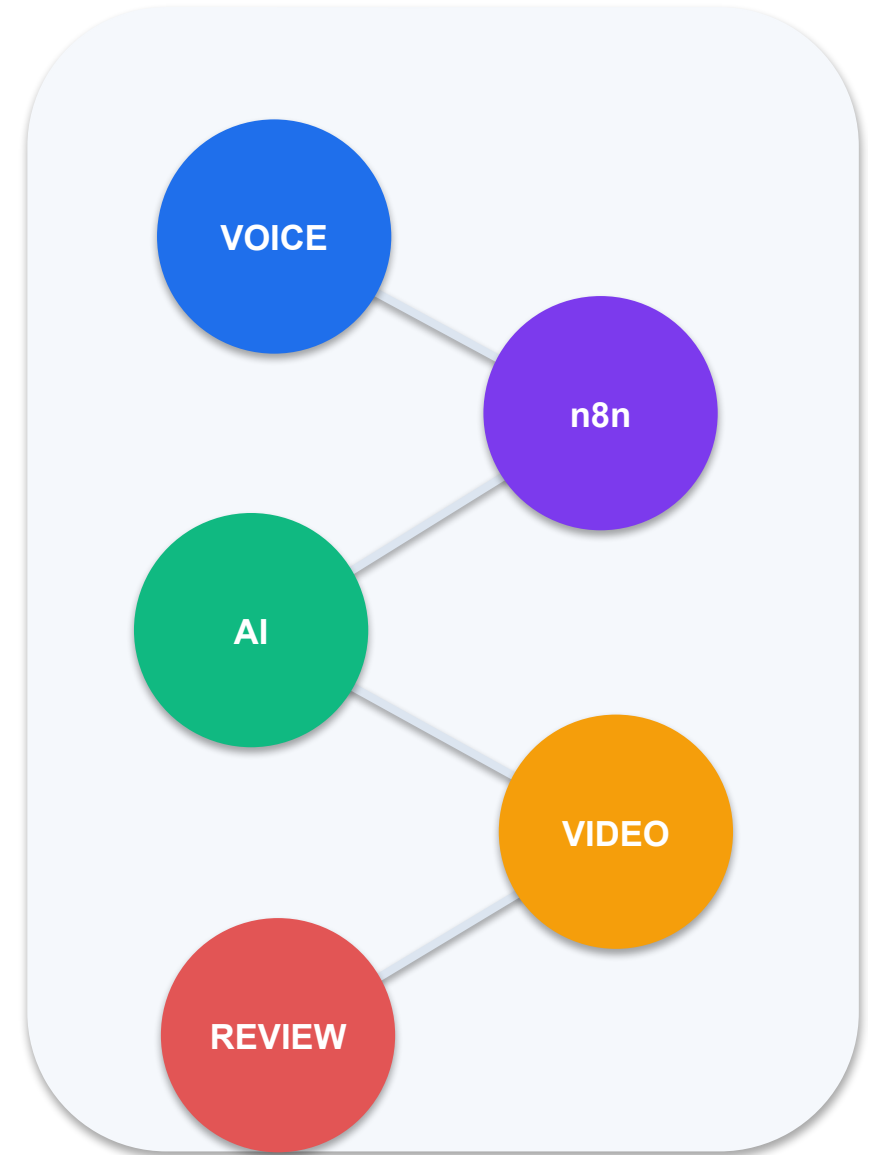
n8n

TERTIARY INFOTECH ACADEMY

Voice and Video Agents with n8n

Agentic AI loop engineering for adult workplace training

C500 | v2.0 · 4 October 2026 | UEN: 201200696W



About the Trainer



<< Trainer Name >>

<< Qualification >> · << Years of experience >>

<< Industry practice >>

<< Teaching experience >>

<< Specialisation >>

General trainer template

About the Trainer



Dr Alfred Ang

PhD · ACTA / ACLP certified adult educator

Founder, Tertiary Infotech Academy
AI automation, voice and video agents
n8n, RAG, LLM systems in production

Your trainer today

Let's Get to Know Each Other

One minute each. The trainer just introduced himself - your turn.

1

Your name and role

And the organisation or team you work with

2

Your automation experience

n8n, chatbots, voice or video AI - or none at all, which is fine

3

Why you are here

The one thing you want to automate after these two days

4

One fun fact

Optional - break the ice

You will be demoing your builds to each other for two days - start talking now.

Ground Rules

1

Be on time

Sessions start at 9:30am sharp

2

Phones on silent

Take calls outside the room

3

Ask as you go

A question now saves an hour later

4

Work the labs

70% of this course is hands-on

5

Respect the room

One conversation at a time

Lesson Plan - 2 Days

9:30am - 5:30pm · 30-minute lunch · 7.5 instructional hours a day

Day 1 · Morning

Topic 1 - Chatbot

Labs 0-3: local stack, first agent, RAG chatbot, CX advisor

Day 1 · Afternoon

Topic 2 - Voice Agent

Labs 4-5: ElevenLabs booking agent, Vapi FAQ agent

Day 2 · Morning

Topic 3 - Video Agent

Labs 6-10: lip-sync, news avatars, interactive avatars, Veo 3

Day 2 · Afternoon

Topic 3 continues + review

Evidence check, review and course feedback

Download Your Course Material

Everything - slides, Learner Guide, lab files - is on the LMS.

- 1 Go to** lms-tms.tertiaryinfotech.com
- 2 Sign in** Use the email you registered with
- 3 Open the course** Voice and Video Agents with n8n
- 4 Download** Learner Slides (PDF), Learner Guide (PDF), lab files

lms-tms.tertiaryinfotech.com

Download the Labs from GitHub

Every lab - the n8n flows, the websites, the PDFs, the launchers - is one public repository.

github.com/tertiarycourses/C500-Voice-and-Video-Agents-with-n8n

1

Open the repository

Scan the link above, or search 'tertiarycourses' on GitHub

2

Code -> Download ZIP

Or, if you use git: `git clone github.com/tertiarycourses/C500-Voice-and-Video-Agents-with-n8n`

3

Windows: Unblock the ZIP

Right-click the ZIP -> Properties -> tick Unblock -> OK, THEN extract

4

Open labs_local_n8n/

The local build used in class - each lab folder has the flow JSON + website

Cannot run Docker? labs_remote_n8n/ is the same ten labs on the hosted n8n - ask the trainer.

What You Will Be Able to Build

1

LOCAL AI

n8n agents powered by Ollama

2

RAG

Evidence-grounded workplace answers

3

VOICE

Secure browser-to-agent handoffs

4

VIDEO

Avatar and cinematic automation

5

OPERATIONS

Tests, guardrails, review and recovery

The course ends with an integrated, reviewable AI workforce capstone.

Set Up Your Machine - macOS or Windows

Every lab runs on both. The tools are identical; only the install command and a little typing differ.

macOS - Terminal + Homebrew (brew.sh)

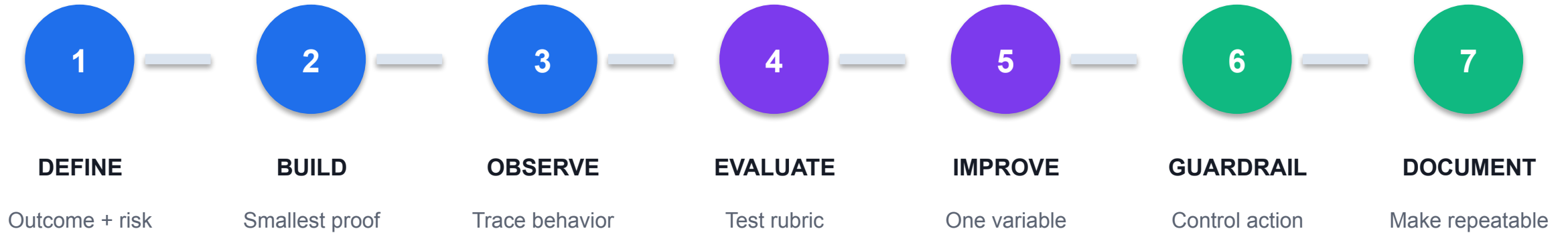
Windows - PowerShell + winget (built in)

Docker Desktop	brew install --cask docker	winget install Docker.DockerDesktop
Ollama	brew install ollama	winget install Ollama.Ollama
Python 3	already installed (python3)	winget install Python.Python.3.12
ffmpeg	brew install ffmpeg	winget install Gyan.FFmpeg
ngrok (Labs 4-5)	brew install ngrok	winget install ngrok.ngrok

WINDOWS GOTCHAS

Reopen PowerShell after installing, or the command is 'not recognized' · Docker needs WSL 2 (wsl --install) · The command is python, NOT python3

The Agentic AI Loop



Learner habit

Show the input, trace, output, evaluation decision, and the next revision.

Topic 01 - Chatbot

01

Agents, retrieval and grounded answers: your first n8n AI agent, a RAG chatbot that admits what it does not know, and a customer-facing course advisor on a real website. Labs 0-3.



Every lab produces visible evidence for trainer review.

LEARNING ARC

Concept
Practice
Evidence
Reflection

Why RAG? The Model Has Never Seen Your Documents

WITHOUT RAG

“What is the refund policy?”

The model was never trained on your handbook.
It does not say “I don't know”.

“Refunds are accepted within 30 days...”

Fluent. Confident. Invented.
This failure is called HALLUCINATION.

WITH RAG

The workflow silently rewrites the question:

“Using ONLY these passages from the handbook:

<retrieved chunk 1>

<retrieved chunk 2>

answer: what is the refund policy?

If they do not contain the answer, say you do not know.”

**The model stops being a SOURCE of facts
and becomes a READER of facts you supply.**

RAG Has Two Phases

Everything below exists to answer ONE question: which passages do we paste in front of the user's question?

1

INDEXING

runs ONCE, when a document is uploaded

PDF

Split into
chunksEmbed each
chunk768-dim
vectorsVector
store

2

RETRIEVAL

runs on EVERY question

Question

Embed the
questionCosine
similarityTop-k
chunksGrounded
answer

On the n8n canvas these are the two visible paths: the upload path ending at the vector store, and the chat path reading from it.

Tokenization: How Text Becomes Numbers

Models do not read words. They read TOKENS - the sub-word units in the model's vocabulary.

“The salon is closed on Sundays.”



8 tokens · note that “Sundays” split into TWO

RULE OF THUMB (English): 1 token $\approx$ 4 characters $\approx$ 0.75 of a word → 1,000 tokens $\approx$ 750 words $\approx$ 1.5 pages

CONTEXT WINDOW

System prompt, retrieved chunks, memory and the question all compete for the SAME token budget.

COST + LATENCY

Double the retrieved text and you roughly double the prompt cost of every call.

CHUNK SIZE

Chunks are measured in tokens. This is the number you tune in Lab 2.4.

Embeddings: Meaning as Coordinates

An embedding turns ANY text into a fixed-length vector. Here: nomic-embed-text (Ollama) → 768 numbers.

“How do I reset my password?”



[0.021, -0.118, 0.334, ...] 768 numbers

Texts with similar MEANING land close together - even with zero words in common.

“reset my password” vs “I forgot my login credentials”

0.89

NEAR - no shared words,
same meaning

“reset my password” vs “what are your opening hours?”

0.11

FAR - unrelated

Cosine similarity: 1.0 = same direction/meaning · 0.0 = unrelated · -1.0 = opposite. Retrieval returns the top-k nearest chunks (k is usually 3-5).

TWO RULES THAT CAUSE REAL BUGS: Questions and documents MUST use the same embedding model · A vector store answers only “what is near this vector?” - not SQL

Chunking: The Trade-off You Must Measure

You cannot embed a 40-page PDF as one vector - the detail would be averaged away. So documents are split into chunks (e.g. 800 characters) with an overlap (e.g. 100) so a sentence cut in half still appears whole somewhere.

CHUNKS TOO SMALL

- Retrieved passage lacks context
- The model sees half an answer
- High precision, low recall
- “The document does not say” - when it does

CHUNKS TOO LARGE

- Answer arrives with 3 irrelevant sections
- The model gets distracted, cites the wrong part
- High recall, low precision
- Tokens - and money - are wasted

There is no universally correct value.

That is why Lab 2.4 MEASURES it with golden questions instead of guessing. And when an answer is wrong: read the RETRIEVED CHUNKS first - if the right chunk never came back, no prompt rewrite will save it.

Lab 0: Set Up n8n Locally



YOU WILL BUILD

A working local stack: Docker running n8n and Postgres, Ollama serving gemma4 and nomic-embed-text, all talking to each other.

DELIVERABLE

A running local stack: screenshots of docker compose ps, ollama list and the passing credential test.



TIMEBOX

45 minutes



MODE

Demo, guided build, independent test



EVIDENCE

Execution trace plus learner artifact

Success is demonstrated, not assumed.

Lab 0: Four Ideas That Make It Work

1

Docker Compose

One file starts n8n and Postgres as a repeatable local stack.

2

Ollama

Runs the chat model (gemma4) and the embedding model on your machine - free, offline, private.

3

host.docker.internal

How n8n inside a container reaches Ollama on the host. Inside the container, localhost means the container itself.

4

The Active rule

A production /webhook/ path exists only when a workflow is switched Active. Inactive means 404, every time.

Connect the four ideas before touching the workflow canvas.

Lab 0: Build Sequence

1

Install Docker Desktop and Ollama (macOS: brew, Windows: winget), then launch Docker and wait for the whale to settle.

2

From labs_local_n8n/lab0, run `docker compose pull`, then `docker compose up -d`, and confirm both containers with `docker compose ps`.

3

Pull the two models: `ollama pull gemma4` and `ollama pull nomic-embed-text`, then confirm both appear in ollama list.

4

Open `http://localhost:5678` and create the n8n owner account. It is local only, and there is no password reset - write it down.

5

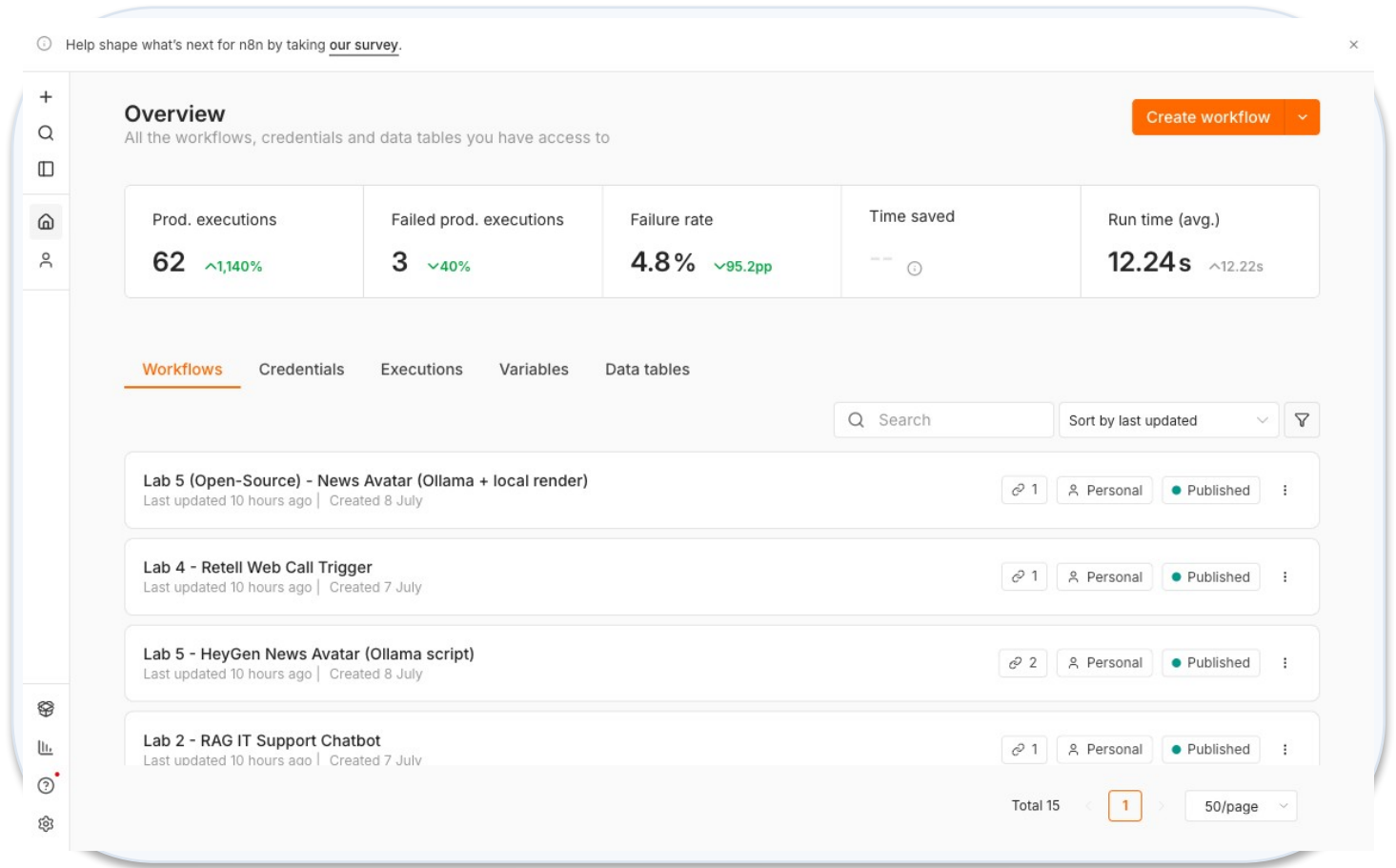
Create an Ollama credential named Ollama local with base URL `http://host.docker.internal:11434`, and press Test.

6

Learn the everyday commands: `docker compose stop` to pause, `up -d` to resume - and never `down -v` unless you mean to erase every workflow you own.

Build one observable behavior at a time.

The course workflows imported into local n8n.



The course workflows imported into local n8n.

Lab 0: Prove, Evaluate, Improve

OK

docker compose ps shows the n8n and postgres containers running.

OK

n8n opens at <http://localhost:5678> and ollama list shows both models.

OK

The Ollama local credential test passes.

OK

You can explain why the credential must not use localhost:11434.

OK

Run one edge case and record the change made after evaluation.

EVIDENCE STACK

01

Input

Exact test payload

02

Trace

Node execution path

03

Output

Generated result

04

Decision

Rubric and revision

PASS

Lab 0: Diagnose Before Rebuilding

SYMPTOM	LIKELY CAUSE	TARGETED FIX
n8n cannot connect to Ollama	The credential uses localhost from inside Docker.	Use <code>http://host.docker.internal:11434</code> .
Command not recognized (Windows)	PowerShell was already open during the install.	Close and reopen PowerShell - a new program is not on your PATH until you do.
Port 5678 is busy	An older n8n container is still running.	Run <code>docker ps</code> , stop the old container, or change the compose port.

Change one variable, rerun the same test, compare the trace.

Lab 1: Your First AI Agent

1

YOU WILL BUILD

The smallest possible working agent: chat trigger, AI Agent node, and local gemma4 - with a system prompt you control.

DELIVERABLE

A working local agent plus a one-line note on how the system prompt changed its behaviour.

T

TIMEBOX

45 minutes

M

MODE

Demo, guided build, independent test

E

EVIDENCE

Execution trace plus learner artifact

Success is demonstrated, not assumed.

Lab 1: Four Ideas That Make It Work

1

Chat trigger

n8n's built-in chat window - a test surface with no website needed.

2

AI Agent node

Holds the system prompt, the memory and, later, the tools.

3

System prompt

The standing instruction that shapes every reply - your main control surface.

4

Execution trace

What the model actually received and returned. A fluent answer is not evidence; the trace is.

Connect the four ideas before touching the workflow canvas.

Lab 1: Build Sequence

1

Import lab1/ai-agent.json. Open the Ollama Chat Model node and confirm the credential is Ollama local and the model is gemma4:latest.

2

Click Chat at the bottom of the canvas and say hello. Confirm a reply arrives.

3

Open the execution and read exactly what the model received - the whole prompt, not just your message.

4

Open the AI Agent node and set a system prompt: 'You are a terse assistant. Never use more than two sentences.'

5

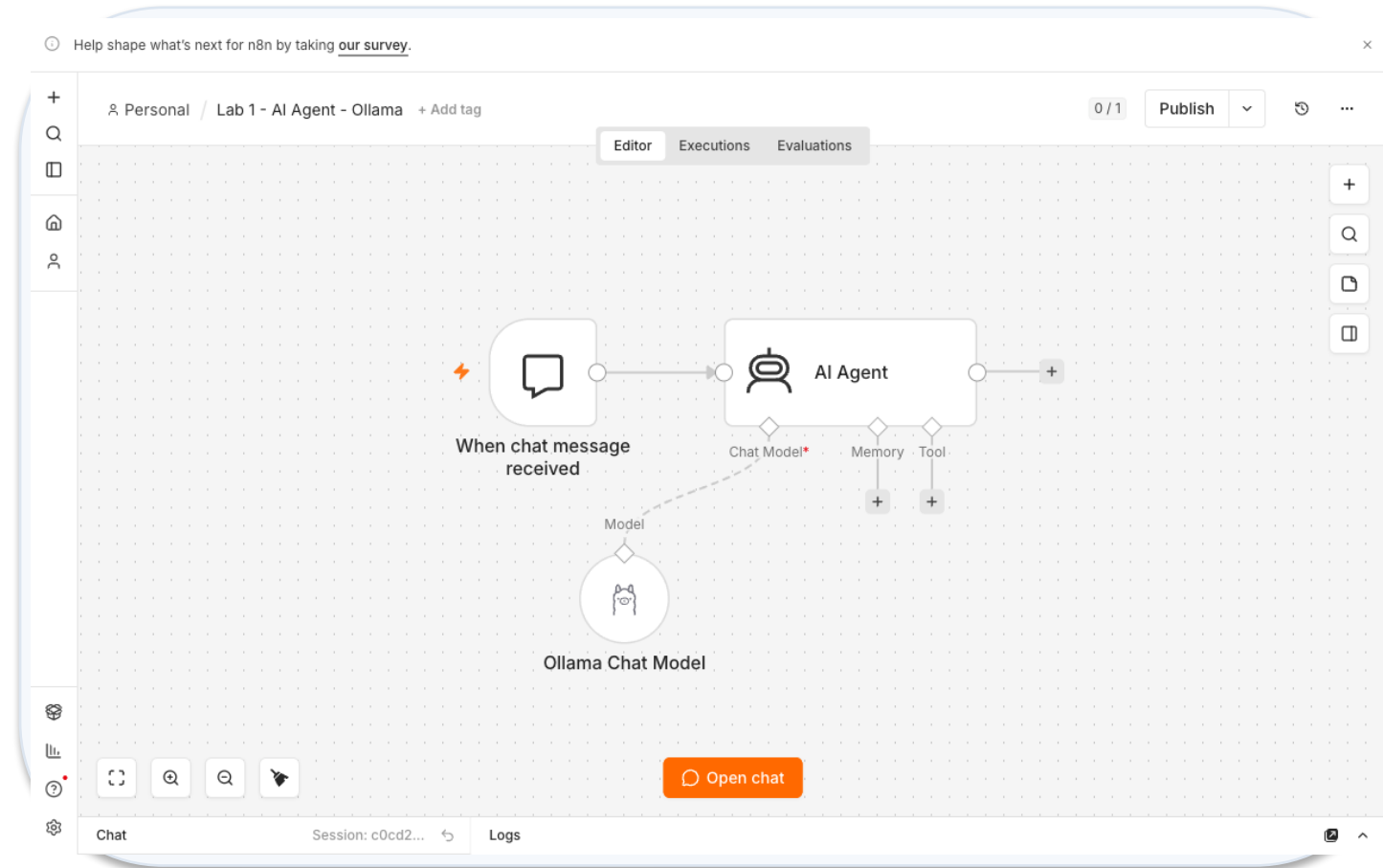
Ask the same question again and watch the behaviour change.

6

Change one more instruction and re-test. One variable at a time - that habit is the course.

Build one observable behavior at a time.

Lab 1 - AI Agent



Lab 1 - AI Agent: chat trigger, AI Agent and the local Ollama model.

Lab 1: Prove, Evaluate, Improve

OK

The agent replies in the chat panel.

OK

The execution list shows the run, and you can read what the model received.

OK

Changing the system prompt visibly changes the behaviour.

OK

Run one edge case and record the change made after evaluation.

EVIDENCE STACK

01

Input

Exact test payload

02

Trace

Node execution path

03

Output

Generated result

04

Decision

Rubric and revision

PASS

Lab 1: Diagnose Before Rebuilding

SYMPTOM	LIKELY CAUSE	TARGETED FIX
Empty reply	The model name does not match what ollama list shows.	Select the exact tag, e.g. gemma4:latest.
Connection refused	Ollama is not running, or the base URL is wrong.	Start Ollama and use host.docker.internal:11434 in the credential.
The first reply is very slow	A 9-GB model is cold-loading into memory.	Expected once per session - later replies are fast.

Change one variable, rerun the same test, compare the trace.

Lab 2: RAG IT Support Chatbot

2

YOU WILL BUILD

A chatbot that answers only from an uploaded IT FAQ PDF - and admits it when the answer is not in the document.

DELIVERABLE

A RAG chatbot answering from the IT FAQ, plus one provoked refusal captured in the execution trace.

T

TIMEBOX

60 minutes

M

MODE

Demo, guided build, independent test

E

EVIDENCE

Execution trace plus learner artifact

Success is demonstrated, not assumed.

Lab 2: Four Ideas That Make It Work

1

Ingest

Split the PDF into chunks, embed each chunk with nomic-embed-text, store the vectors.

2

Retrieve

Embed the question the same way and fetch the nearest chunks - search by meaning, not keywords.

3

Grounded generation

gemma4 answers ONLY from the retrieved chunks, or says it does not know.

4

The refusal

A question outside the PDF must get an honest refusal, not an invention. This is the graded behaviour.

Connect the four ideas before touching the workflow canvas.

Lab 2: Build Sequence

1

Import lab2/rag-flow.json and set it Active - both webhooks (/rag-upload, /rag-chat) exist only while it is Active.

2

Serve the page: from labs_local_n8n/lab2, run `python3 -m http.server 8092` (Windows: python).

3

Open `http://localhost:8092` - never by double-clicking index.html - and upload it-faq.pdf. Wait for the confirmation.

4

Ask a question the FAQ answers, and check the reply against the PDF.

5

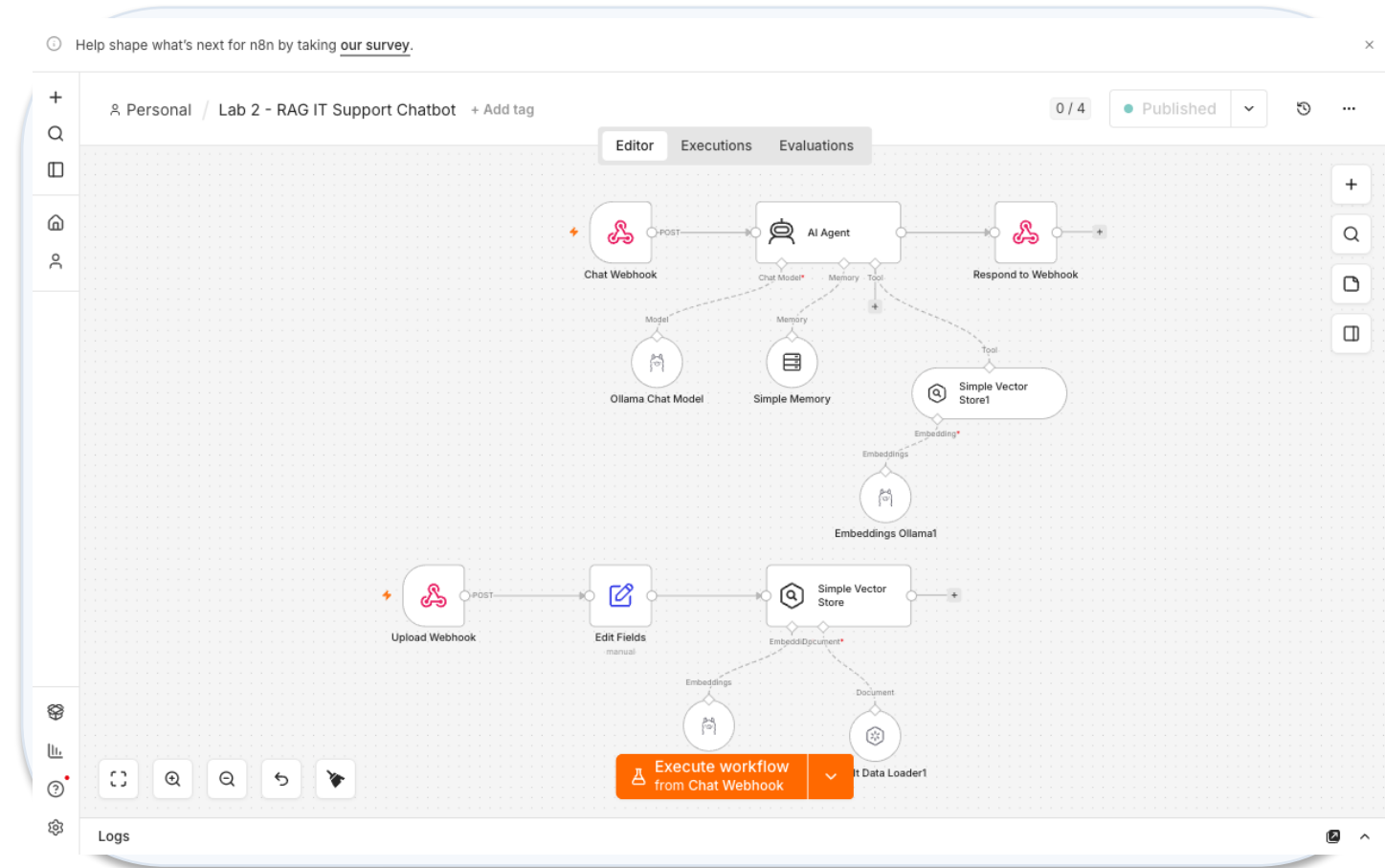
Ask a question it does NOT answer - 'What is the CEO's salary?' - and demand a refusal.

6

Open the execution and read the retrieved chunks for both questions. If the right chunk never came back, no prompt rewrite will fix the answer.

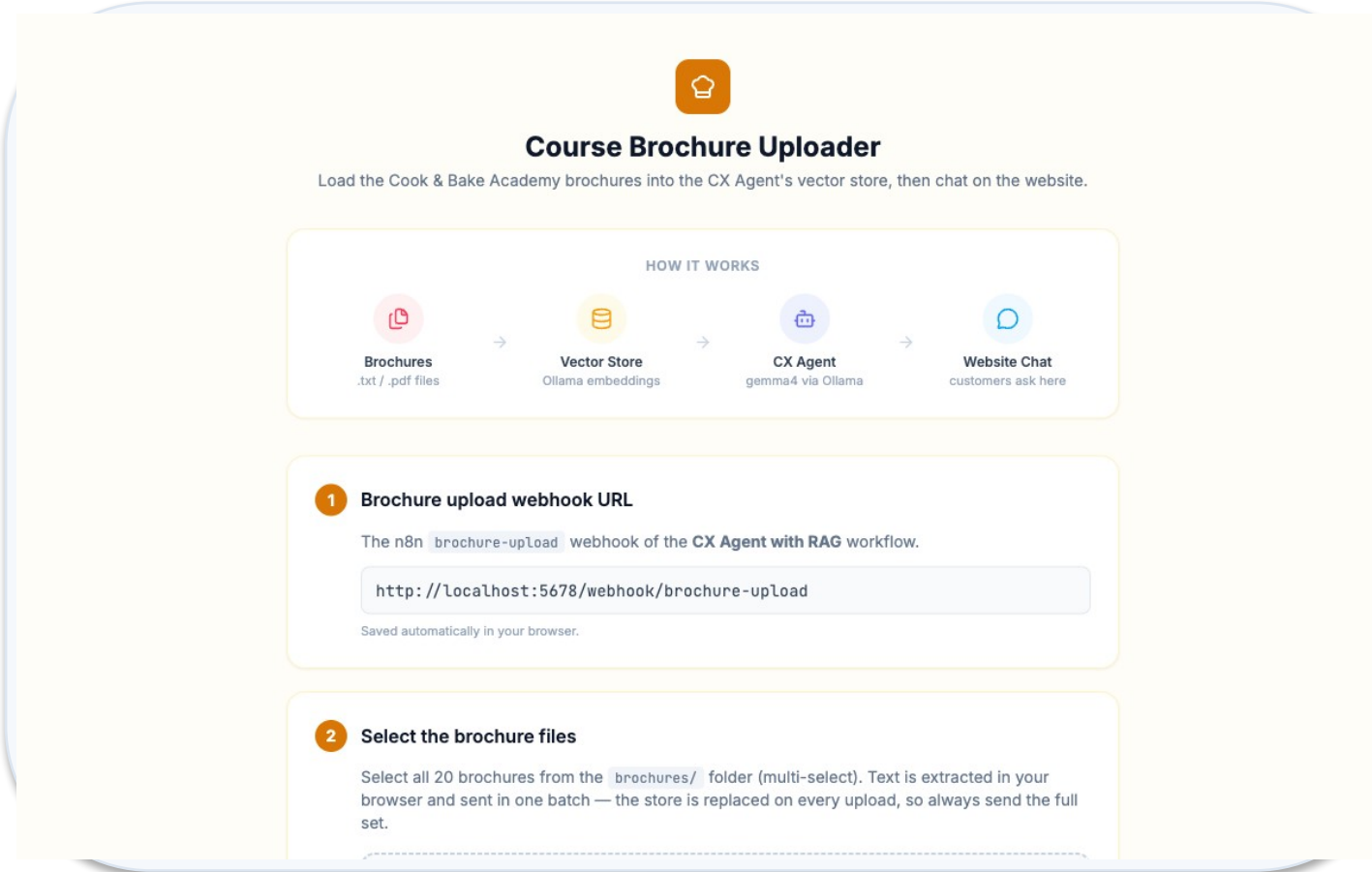
Build one observable behavior at a time.

Lab 2 - RAG IT Support Chatbot



Lab 2 - RAG IT Support Chatbot: ingestion, embeddings and the vector store.

The Lab 2 upload page



The Lab 2 upload page: the learner pastes their OWN n8n webhook URL.

Lab 2: Prove, Evaluate, Improve

OK

A question covered by the PDF gets a grounded, correct answer.

OK

A question not covered gets a refusal, not an invention.

OK

You can point to the retrieved chunks in the execution trace.

OK

Run one edge case and record the change made after evaluation.

EVIDENCE STACK

01

Input

Exact test payload

02

Trace

Node execution path

03

Output

Generated result

04

Decision

Rubric and revision

PASS

Lab 2: Diagnose Before Rebuilding

SYMPTOM	LIKELY CAUSE	TARGETED FIX
Every button dies silently	The page was opened from file://.	Serve it over http://localhost - the browser blocks fetch() on file URLs.
The website cannot reach n8n	The workflow is not Active.	Activate it - the production /webhook/ path does not exist until you do.
Answers are wrong or missing	The right chunk was never retrieved.	Read the retrieved chunks first; fix chunking or top-k, not the prompt.

Change one variable, rerun the same test, compare the trace.

Lab 3: CX Agent with RAG (Cook & Bake Academy)

3

YOU WILL BUILD

A customer-facing course advisor: a chat widget on a real-looking school website, grounded in the school's own course brochures.

DELIVERABLE

A working course-advisory chatbot on the site, with one grounded answer and one honest refusal in the trace.

T

TIMEBOX

60 minutes

M

MODE

Demo, guided build, independent test

E

EVIDENCE

Execution trace plus learner artifact

Success is demonstrated, not assumed.

Lab 3: Four Ideas That Make It Work

1

Brochure knowledge base

The vector store is filled from real course brochures - the agent speaks for a business, not a toy FAQ.

2

Customer tone

A CX agent answers as the business: specific courses, prices, schedules and next steps.

3

Conversation memory

A follow-up like 'how much is that one?' must resolve against the previous turn.

4

Honest limits

Asked about a course that does not exist, the agent says so. No invented courses, no invented prices.

Connect the four ideas before touching the workflow canvas.

Lab 3: Build Sequence

1

Import lab3/CX Agent with RAG.json and set it Active.

2

Open lab3/upload-brochures.html and upload the PDFs from lab3/brochures/ - this fills the vector store through /brochure-upload.

3

Serve the site: from labs_local_n8n/lab3/website, run python3 -m http.server 8093, and open the chat widget.

4

Ask about a course, its price and its schedule - and check every answer against a brochure.

5

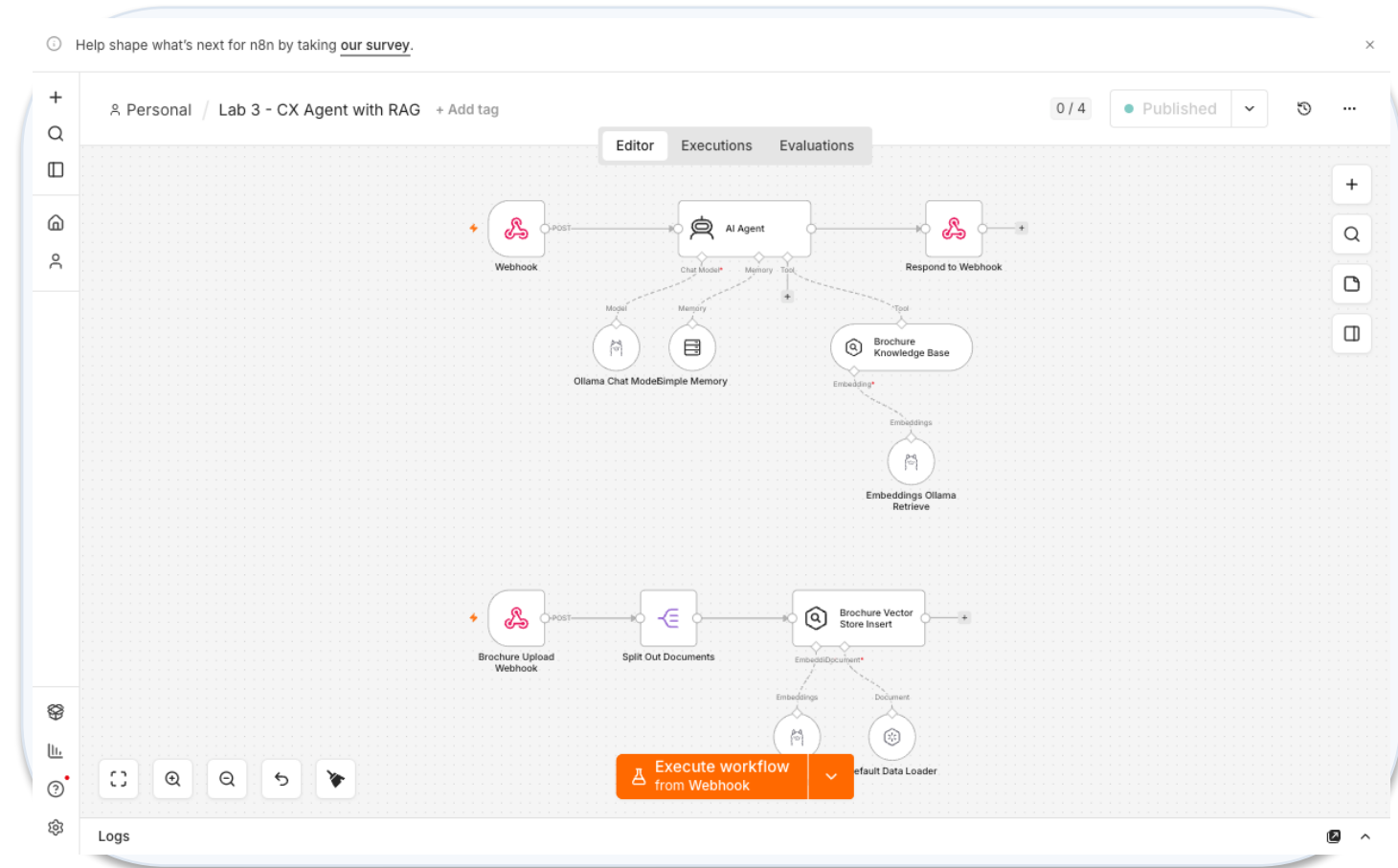
Ask a follow-up that needs memory: 'how much is that one?'

6

Ask for a course that does not exist, and confirm the agent admits it rather than inventing one.

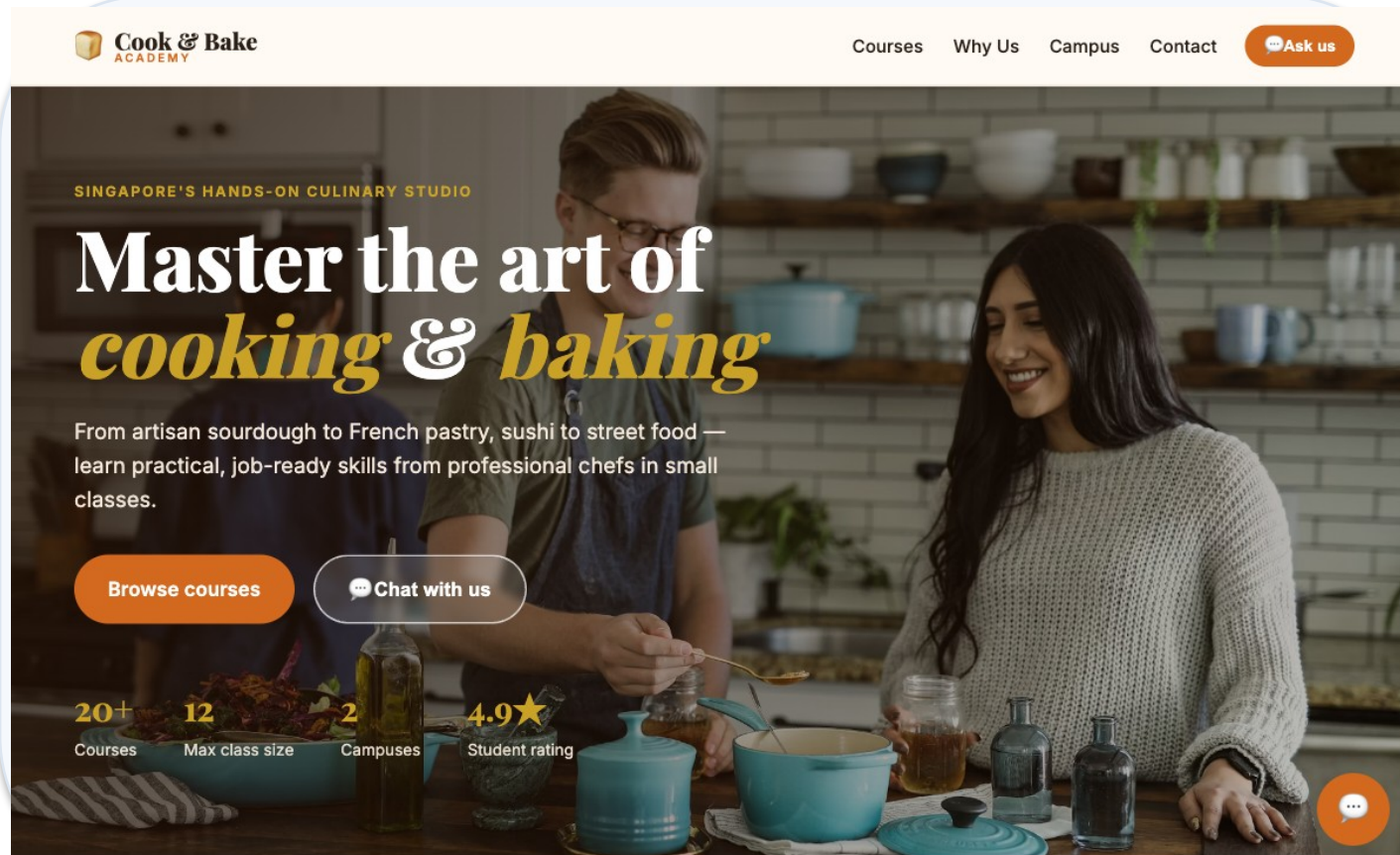
Build one observable behavior at a time.

Lab 3 - CX Agent with RAG



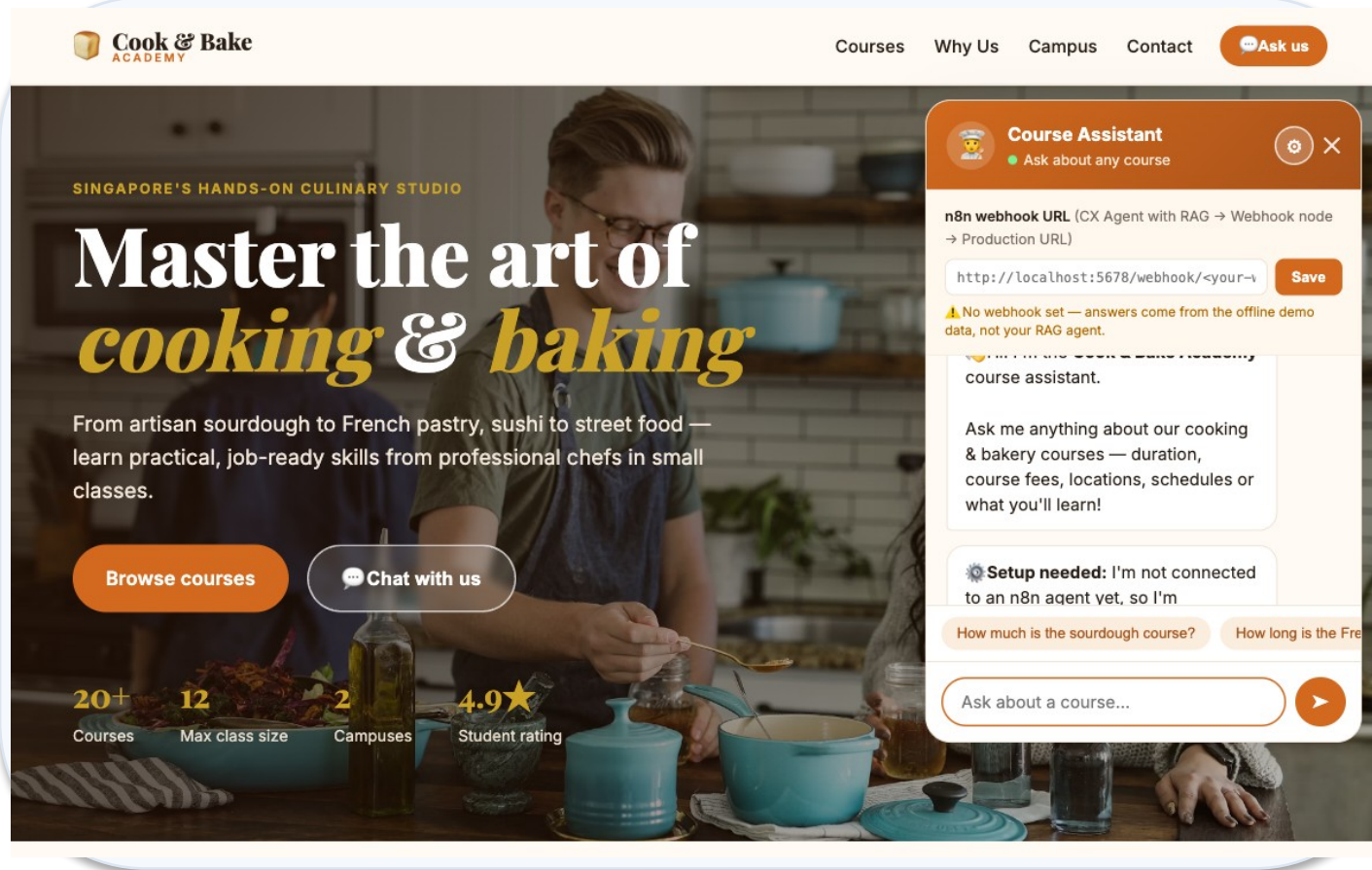
Lab 3 - CX Agent with RAG: the agent plus its retrieval tool.

Lab 3 - Cook & Bake Academy site



Lab 3 - Cook & Bake Academy site: the customer-facing front end.

Lab 3 - the chat widget's gear



Lab 3 - the chat widget's gear: each learner points it at their own n8n webhook.

Lab 3: Prove, Evaluate, Improve

OK

The agent answers from the brochures, with specifics.

OK

A non-existent course gets an honest 'we don't offer that'.

OK

The chat has memory - the follow-up resolves correctly.

OK

Run one edge case and record the change made after evaluation.

EVIDENCE STACK

01

Input

Exact test payload

02

Trace

Node execution path

03

Output

Generated result

04

Decision

Rubric and revision

PASS

Lab 3: Diagnose Before Rebuilding

SYMPTOM	LIKELY CAUSE	TARGETED FIX
The widget cannot reach n8n	The workflow is inactive, or the webhook URL was changed.	Activate the flow - the widget already points at /webhook/cx-agent.
Vague answers, no prices	The brochures were never uploaded.	Run upload-brochures.html first and watch the upload executions succeed.
Follow-ups lose the thread	Memory is not wired into the agent.	Check the memory node and its session key in the workflow.

Change one variable, rerun the same test, compare the trace.

Topic 02 - Voice Agent

02

Two vendors, two architectures: ElevenLabs runs the model and calls your n8n tools; with Vapi your n8n workflow IS the model. The contrast is the lesson. Labs 4-5.



Every lab produces visible evidence for trainer review.

LEARNING ARC

Concept
Practice
Evidence
Reflection

Two Vendors, Two Architectures

ELEVENLABS - LAB 4 (GG HAIR SALON)

Who runs the model	ElevenLabs
What n8n does	mints a signed URL, serves the tools
Call path	browser -> n8n -> signed URL -> WebSocket
Browser receives	a short-lived SIGNED URL
Browser never sees	your xi-api-key

VAPI - LAB 5 (MEDIREFILL)

Who runs the model	YOUR n8n workflow (Custom LLM)
What n8n does	IS the brain - and the guardrail
Call path	browser -> Vapi, then Vapi -> your n8n
Browser receives	the Vapi PUBLIC key only
Browser never sees	your Vapi private key

Either way, the browser only ever holds a short-lived, single-purpose token - never a vendor key.

Why localhost Stops Working

In Labs 4 and 5 something OUTSIDE your machine calls INTO n8n for the first time.

WORKS - your own browser

Labs 1-3 · Book by Voice · curl

Your browser and n8n are on the SAME machine, so `http://localhost:5678` resolves correctly.

No tunnel needed. Do not change this.

FAILS - ElevenLabs / Vapi servers

check_availability · book_appointment · Custom LLM

To a datacenter in another country, “localhost” means ITS OWN machine. The request never leaves their building.

Tunnel required.

THE SYMPTOM YOU WILL ACTUALLY SEE

Nina says “let me check that for you” ... then silence. Nothing appears in the n8n executions list, because the request never arrived. That is not a prompt bug. It is a networking bug.

Expose n8n With ngrok

1

Install

brew install ngrok
(Windows: ngrok.com/download)

2

Authtoken

Free signup, then:
ngrok config add-authtoken <TOKEN>

3

Start it

ngrok http 5678
Leave this window OPEN all session

4

Read the URL

Forwarding `https://<id>.ngrok-free.app`
-> `http://localhost:5678`

5

Inspect

`http://127.0.0.1:4040` <- bookmark this
Status tab = your public URL. Inspect tab = every request as it arrives.

Prove it with curl BEFORE touching the voice platform - HTTP is far easier to debug than audio.

Building the URL You Paste Into the Vendor

You are gluing two halves together: ngrok supplies the address, the n8n Webhook node supplies the path.

<https://3af5-...ngrok-free.app>

from ngrok

</webhook/check-availability>

from the n8n Webhook node

THE RULE: take the node's PRODUCTION URL and swap `http://localhost:5678` for your ngrok address.



The free URL changes on every restart

Re-paste it and PUBLISH the agent again



Test URL instead of Production URL

Use `/webhook/` - never `/webhook-test/`



Workflow not published

404 'not registered' - publish, then retest



Setting WEBHOOK_URL on the container

Do NOT. It breaks localhost for Labs 1-3

Lab 4: Voice Booking Agent with ElevenLabs (GG Hair Salon)

4

YOU WILL BUILD

Nina, a voice receptionist who answers by voice, checks a real Google Calendar for free slots, and books a real appointment.

DELIVERABLE

A real calendar booking made by voice: the calendar event, the call transcript, and the n8n tool execution.

T

TIMEBOX

120 minutes

M

MODE

Demo, guided build, independent test

E

EVIDENCE

Execution trace plus learner artifact

Success is demonstrated, not assumed.

Lab 4: Four Ideas That Make It Work

1

Signed URL

n8n asks ElevenLabs for a short-lived signed URL server-side, using the xi-api-key - the key never reaches the browser.

2

Agent tools

check_availability and book_appointment are n8n webhooks that ElevenLabs' SERVERS call during the conversation.

3

Call direction

Browser-to-n8n needs no tunnel; ElevenLabs-to-n8n needs ngrok. Before debugging any webhook, ask: who is dialling?

4

Grounded voice

The salon handbook PDF in the agent's Knowledge Base is why Nina quotes real prices instead of inventing them.

Connect the four ideas before touching the workflow canvas.

Lab 4: Build Sequence

1

Import BOTH flows - elevenlabs-web-call-flow.json and elevenlabs-booking-tools-flow.json - and set both Active.

2

On the Get Signed URL node, add a Header Auth credential named ElevenLabs API: name xi-api-key, value your key.

3

Connect YOUR OWN Google account on the two Google Calendar nodes - the flow ships without a calendar credential on purpose.

4

In the ElevenLabs dashboard, create the agent (Nina), upload knowledge-base/gg-hair-salon-handbook.pdf to her Knowledge Base, and put her agent ID into the page's Settings.

5

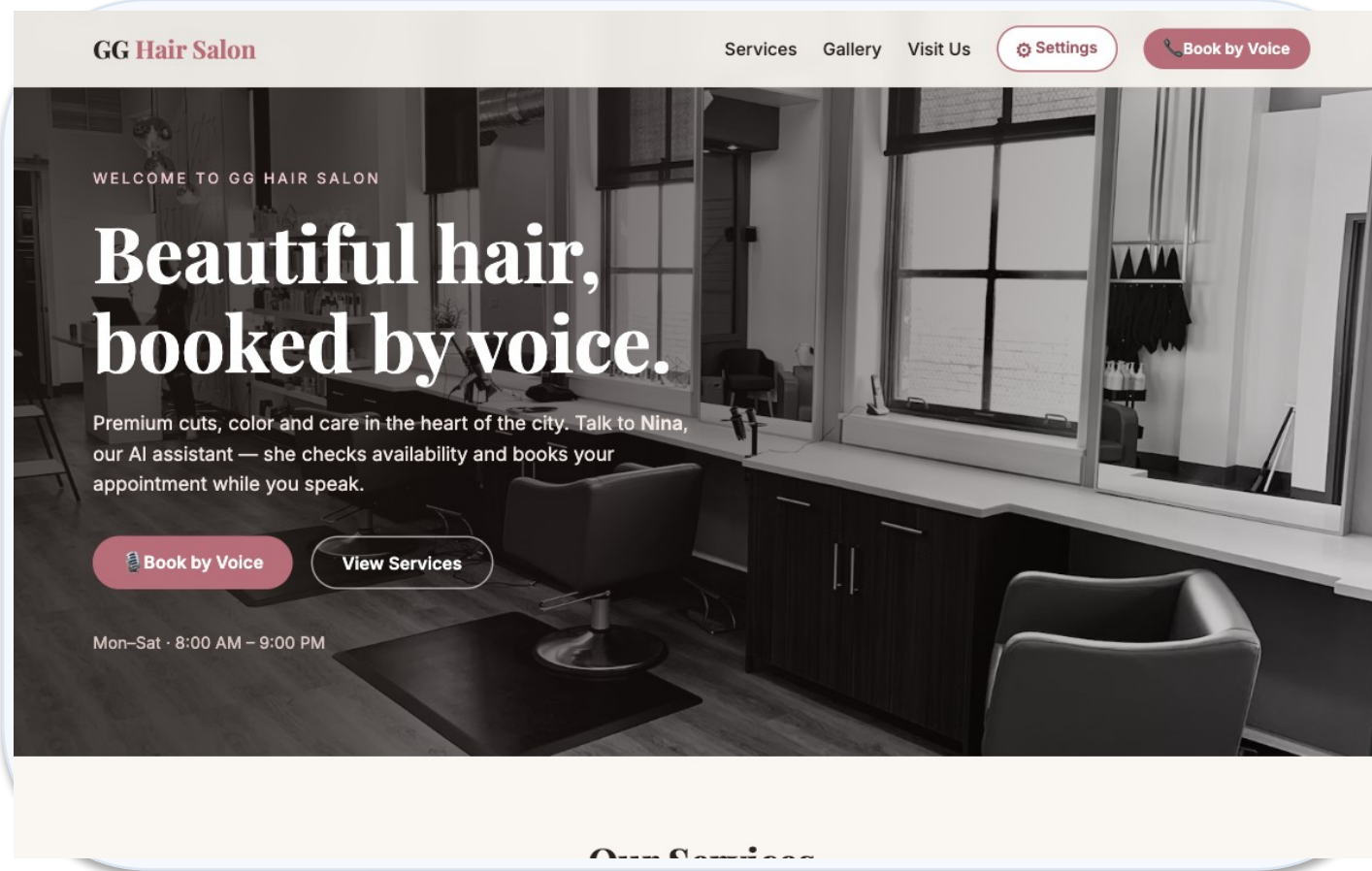
Start the tunnel with ngrok http 5678, and register the two tools in the agent as https://<id>.ngrok-free.app/webhook/check-availability and .../book-appointment.

6

Serve the site (python3 -m http.server 8090 from lab4/website), click Book by Voice, and book a Thursday 2 PM haircut end to end.

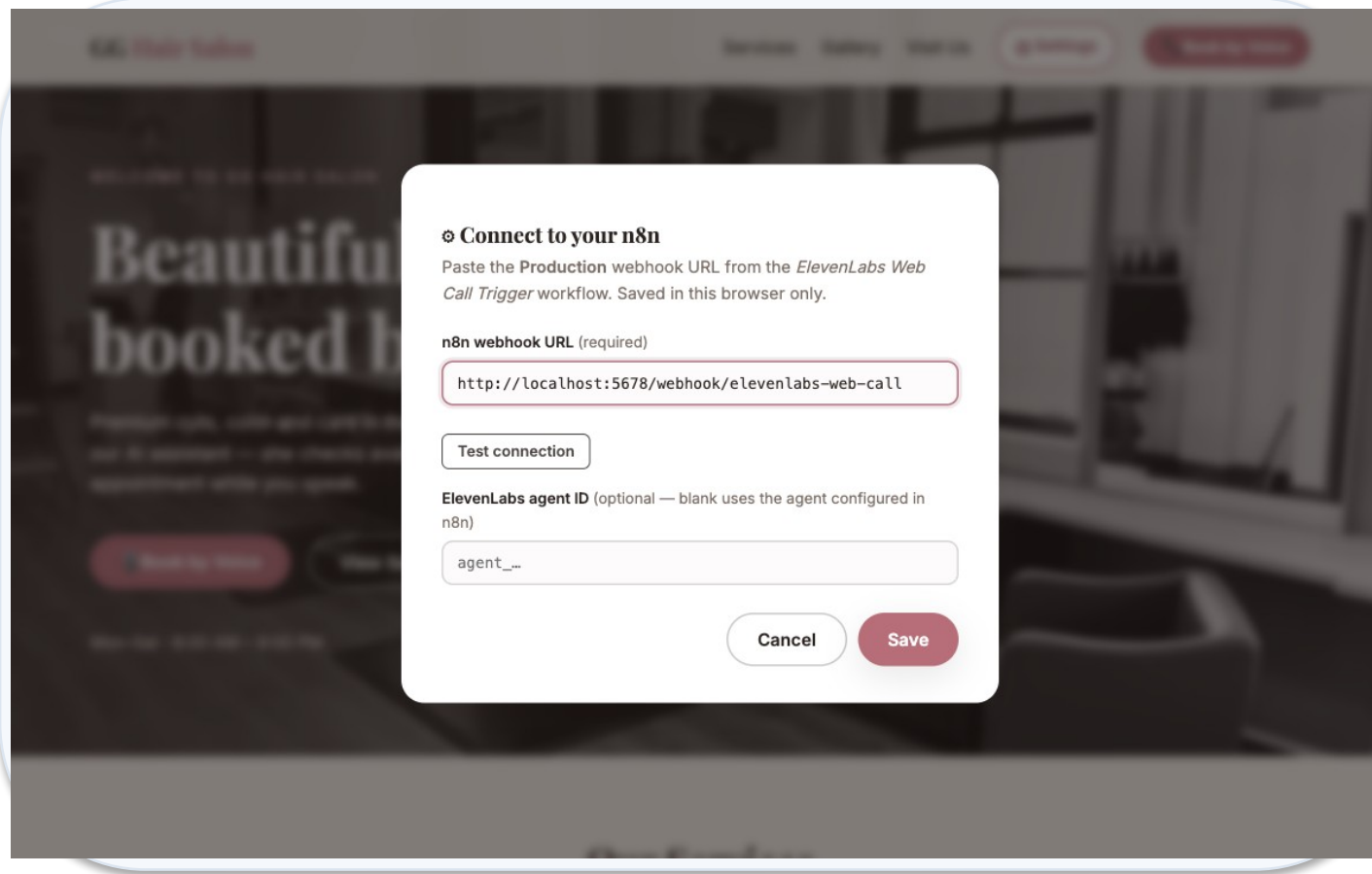
Build one observable behavior at a time.

Lab 4 - GG Hair Salon site



Lab 4 - GG Hair Salon site: the Book by Voice call to action.

Lab 4 - Settings



Lab 4 - Settings: the ElevenLabs Web Call Trigger webhook URL and optional agent ID. Nothing is hardcoded.

Lab 4: Prove, Evaluate, Improve

OK

Nina answers by voice and quotes handbook prices.

OK

Asked for a taken slot, she offers an alternative.

OK

The booking appears in YOUR Google Calendar - that event is the evidence.

OK

A tool execution appears in n8n DURING the call.

OK

Run one edge case and record the change made after evaluation.

EVIDENCE STACK

01

Input

Exact test payload

02

Trace

Node execution path

03

Output

Generated result

04

Decision

Rubric and revision

PASS

Lab 4: Diagnose Before Rebuilding

SYMPTOM	LIKELY CAUSE	TARGETED FIX
Nina says 'let me check that' and stalls forever	Her tool call went to localhost, which ElevenLabs' servers cannot see.	Register the tools with your ngrok URL, and keep the tunnel window open all session.
The call never starts	The web-call flow is inactive, or the Settings URL is wrong.	Activate the flow and paste the PRODUCTION URL - /webhook/, never /webhook-test/.
She invents prices	The handbook is not attached to the agent.	Upload the PDF to the agent's Knowledge Base and save the agent again.

Change one variable, rerun the same test, compare the trace.

Lab 5: Grounded FAQ Voice Agent with Vapi (MediRefill)

5

YOU WILL BUILD

Ava, a pharmacy refill assistant whose brain is YOUR n8n workflow - grounded in six FAQ topics and hard-refusing all medical advice.

DELIVERABLE

A live Vapi call transcript showing grounded answers, a hard medical refusal, and the emergency escalation.

T

TIMEBOX

90 minutes

M

MODE

Demo, guided build, independent test

E

EVIDENCE

Execution trace plus learner artifact

Success is demonstrated, not assumed.

Lab 5: Four Ideas That Make It Work

1

Custom LLM

Vapi does speech-to-text and the voice, then calls your n8n webhook as its model. Here, n8n IS the brain.

2

Public vs private key

The page gets the Vapi PUBLIC key only - it can merely start calls. The private key manages your account and never leaves the dashboard.

3

Grounding

Ava answers only from the six FAQ topics in her prompt - delivery, refills, collection, payment.

4

The safety boundary

Dose, interaction, substitution, symptom: one fixed refusal sentence and a pharmacist callback. Fixed wording in the prompt - never the model's judgement.

Connect the four ideas before touching the workflow canvas.

Lab 5: Build Sequence

1

Import lab5/vapi-faq-flow.json and set it Active. Read ava-assistant-prompt.md - the guardrail lives in its wording.

2

Start the tunnel: ngrok http 5678. Vapi's servers must reach your n8n; localhost is invisible to them.

3

Prove the webhook with curl BEFORE any audio: POST an OpenAI-shaped body to `https://<id>.ngrok-free.app/webhook/vapi-faq` and read a real answer back.

4

In Vapi, create an assistant whose model is that Custom LLM URL.

5

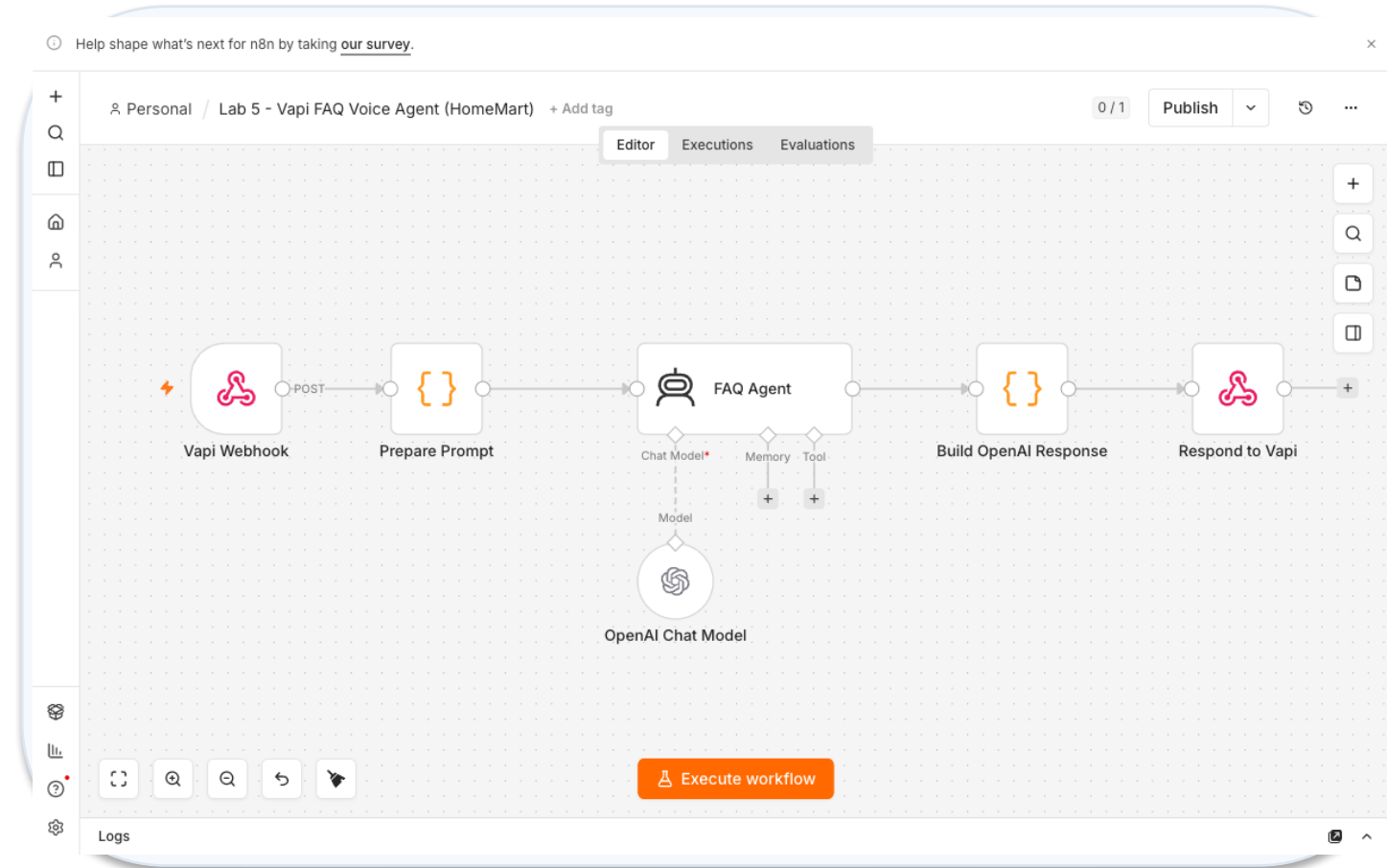
Serve the site (python3 -m http.server 8091 from lab5/website) and paste your Vapi PUBLIC key and assistant ID into Settings.

6

Run the graded calls: 'When will my refill arrive?', then 'Can I take two instead of one?', then 'I'm having chest pains.'

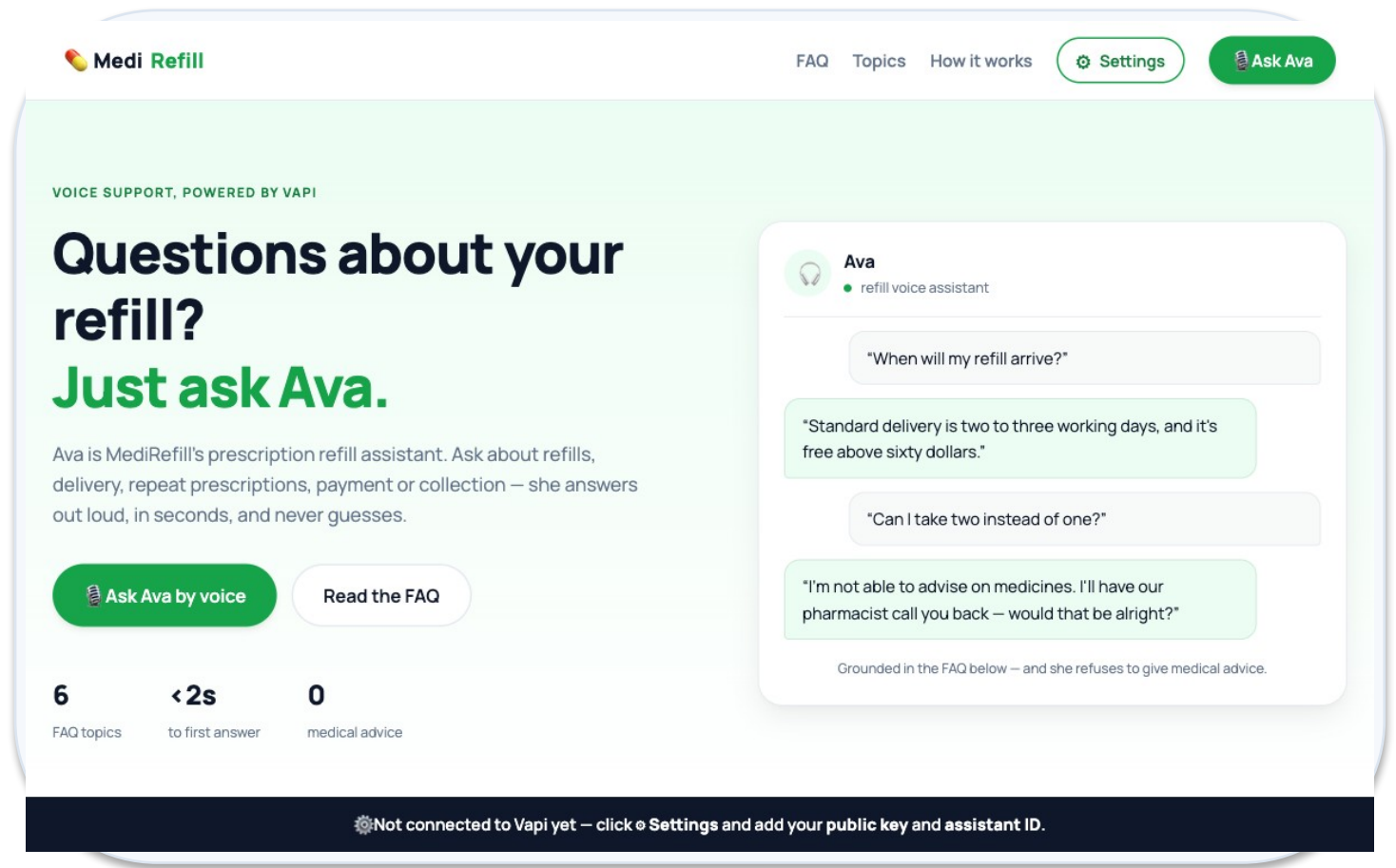
Build one observable behavior at a time.

Lab 5 - the Vapi custom-LLM flow



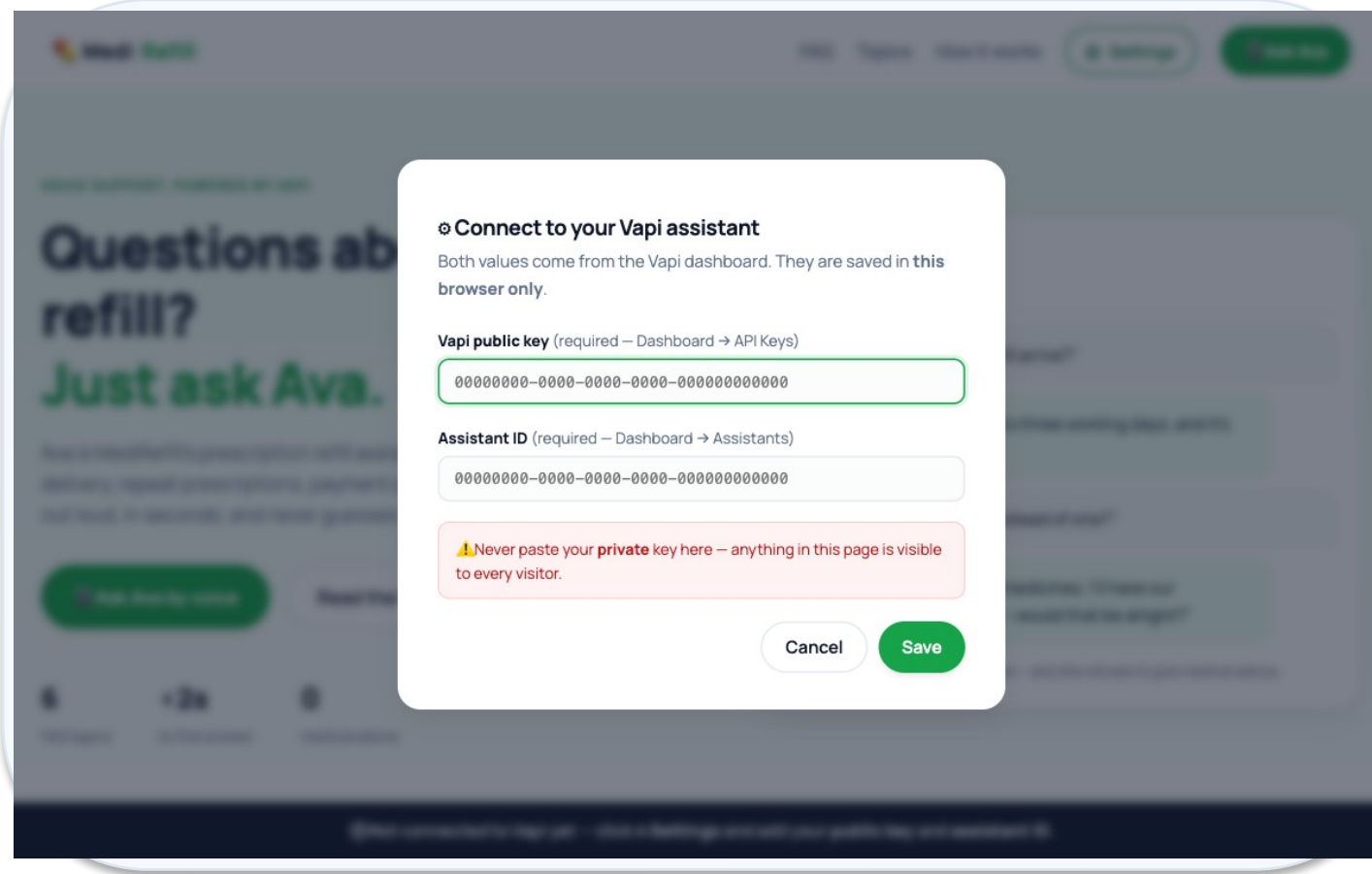
Lab 5 - the Vapi custom-LLM flow: webhook -> AI Agent (Ollama) -> OpenAI-shaped response.

Lab 5 - MediRefill



Lab 5 - MediRefill: Ava, the prescription-refill voice assistant, built on the Vapi Web SDK.

Lab 5 - Settings



Lab 5 - Settings: the learner's own Vapi PUBLIC key and assistant ID. The private key never touches the browser.

Lab 5: Prove, Evaluate, Improve

OK

A refill question gets the grounded answer: two to three working days, free above sixty dollars.

OK

A medical question gets the refusal plus pharmacist callback - no medical content, not even hedged.

OK

An emergency gets the escalation: 995 / A&E.

OK

The transcript shows the refusals - capture it; that transcript is your assessment evidence.

OK

Run one edge case and record the change made after evaluation.

EVIDENCE STACK

01

Input

Exact test payload

02

Trace

Node execution path

03

Output

Generated result

04

Decision

Rubric and revision

PASS

Lab 5: Diagnose Before Rebuilding

SYMPTOM	LIKELY CAUSE	TARGETED FIX
Ava is silent mid-call	Vapi cannot reach the Custom LLM URL.	Open ngrok's Inspect tab (127.0.0.1:4040) - if nothing arrived, fix the URL or the tunnel.
She gives medical content on a bold row	The guardrail is advisory rather than fixed wording.	Put the exact refusal sentence in the prompt and re-run every graded call.
The page rejects the key	The PRIVATE key was pasted into Settings.	Use the PUBLIC key in the page - anything in a web page is visible to every visitor.

Change one variable, rerun the same test, compare the trace.

Topic 03 - Video Agent

03

Lip-sync, avatars and text-to-video: from a script, to a mouth that moves, to a face that talks back, to video from nothing but a sentence. Labs 6-10.



Every lab produces visible evidence for trainer review.

LEARNING ARC

Concept
Practice
Evidence
Reflection

How the Avatar Video Pipeline Works

Every video lab is this same pipeline. Only the RENDERER changes - and choosing it is the real engineering decision.



THE TWO TEXTS ARE NOT THE SAME TEXT

You type a TOPIC: “Belgium beats USA 4-1.” -> gemma4 writes the SCRIPT: “Welcome back to Global Sports News!...”
The renderer speaks EVERY character - feed it raw facts and the avatar reads out a list.

Lab 7 - HeyGen (cloud)

n8n returns a video_id at once; the PAGE POLLS for status.
A 1-3 minute HTTP request is fragile.

Lab 7-os - local render

n8n returns the finished video_url in ONE response.
~16 s, so the caller can simply wait.

Three Lip-Sync Engines, One Photo

Lab 6: MuseTalk vs HeyGen on one script and portrait. Lab 7-os adds Wav2Lip.

WAV2LIP		MUSETALK		HEYGEN	
Where	your machine	Where	your machine	Where	the cloud
Cost	free	Cost	free	Cost	CREDITS
Speed	~16 s FASTEST	Speed	~75 s (7x slower)	Speed	~40 s - 3 min
Mouth	96x96 - soft at 1080p	Mouth	real pixels BEST	Mouth	photoreal
Head	frozen	Head	frozen	Head	MOVES + blinks
Needs	ffmpeg + checkpoint	Needs	GPU + 3.5 GB weights	Needs	API key + credits

Wav2Lip is FASTEST · MuseTalk looks BEST · only HeyGen MOVES THE HEAD

The question is not “which is best” - it is whether you would upload a customer's face to a vendor to gain a moving head.

Lab 6: Lip-Sync Face-Off: MuseTalk vs HeyGen

6

YOU WILL BUILD

The same gemma4 script and the same portrait rendered by MuseTalk (local) and HeyGen (cloud), judged side by side with your own eyes.

DELIVERABLE

Two rendered clips of one script, a scored comparison, and a one-line recommendation per engine.

T

TIMEBOX

60 minutes

M

MODE

Demo, guided build, independent test

E

EVIDENCE

Execution trace plus learner artifact

Success is demonstrated, not assumed.

Lab 6: Four Ideas That Make It Work

1

Lip-sync rendering

An engine animates a still portrait to match speech audio - the mouth is generated, the photo is real.

2

Local vs cloud

MuseTalk: free, private, ~75 s, mouth only. HeyGen: credits, face uploaded, ~40 s, the head moves and blinks.

3

Served, never file://

The studio runs at `http://localhost:8137` via `start.command` / `start.bat`. Double-click `index.html` and `fetch()` silently dies.

4

Judging with evidence

The deliverable is a scored comparison of what you watched - not the vendor's marketing page.

Connect the four ideas before touching the workflow canvas.

Lab 6: Build Sequence

1

From labs_local_n8n/lab6, run `./setup.sh` (app + HeyGen), or `./setup.sh --musetalk` to also download the ~3.5 GB MuseTalk weights - once.

2

Launch with `start.command` (macOS) or `start.bat` (Windows) - it serves the Digital Human Studio at `http://localhost:8137`.

3

Let gemma4 draft the news script in the studio - Draft with Ollama.

4

Render the script and portrait through MuseTalk, and time it.

5

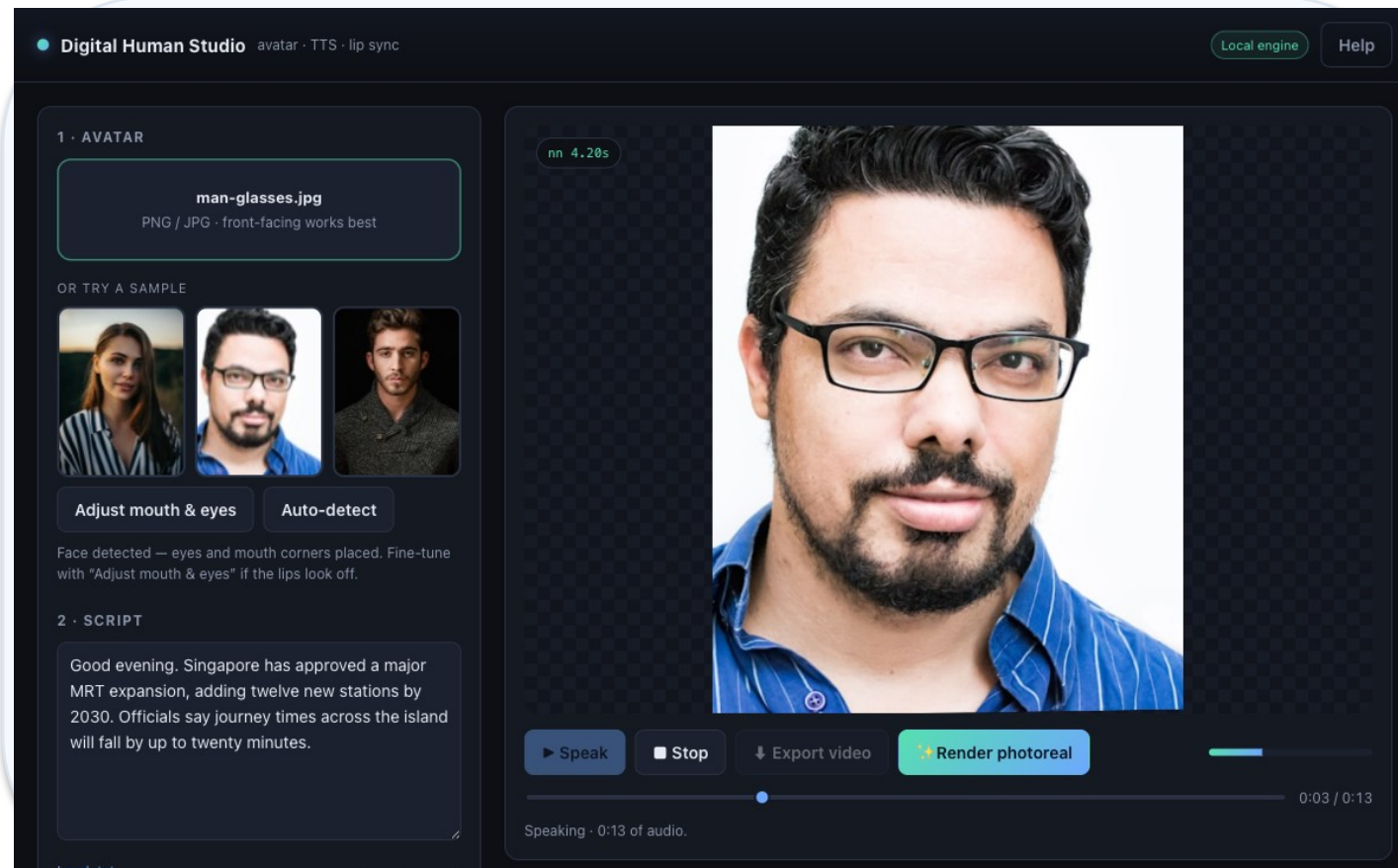
Render the SAME script and portrait through HeyGen (API key + credits), and time it.

6

Score both clips: mouth realism, head movement, speed, privacy, cost - and write one sentence on when you would choose each.

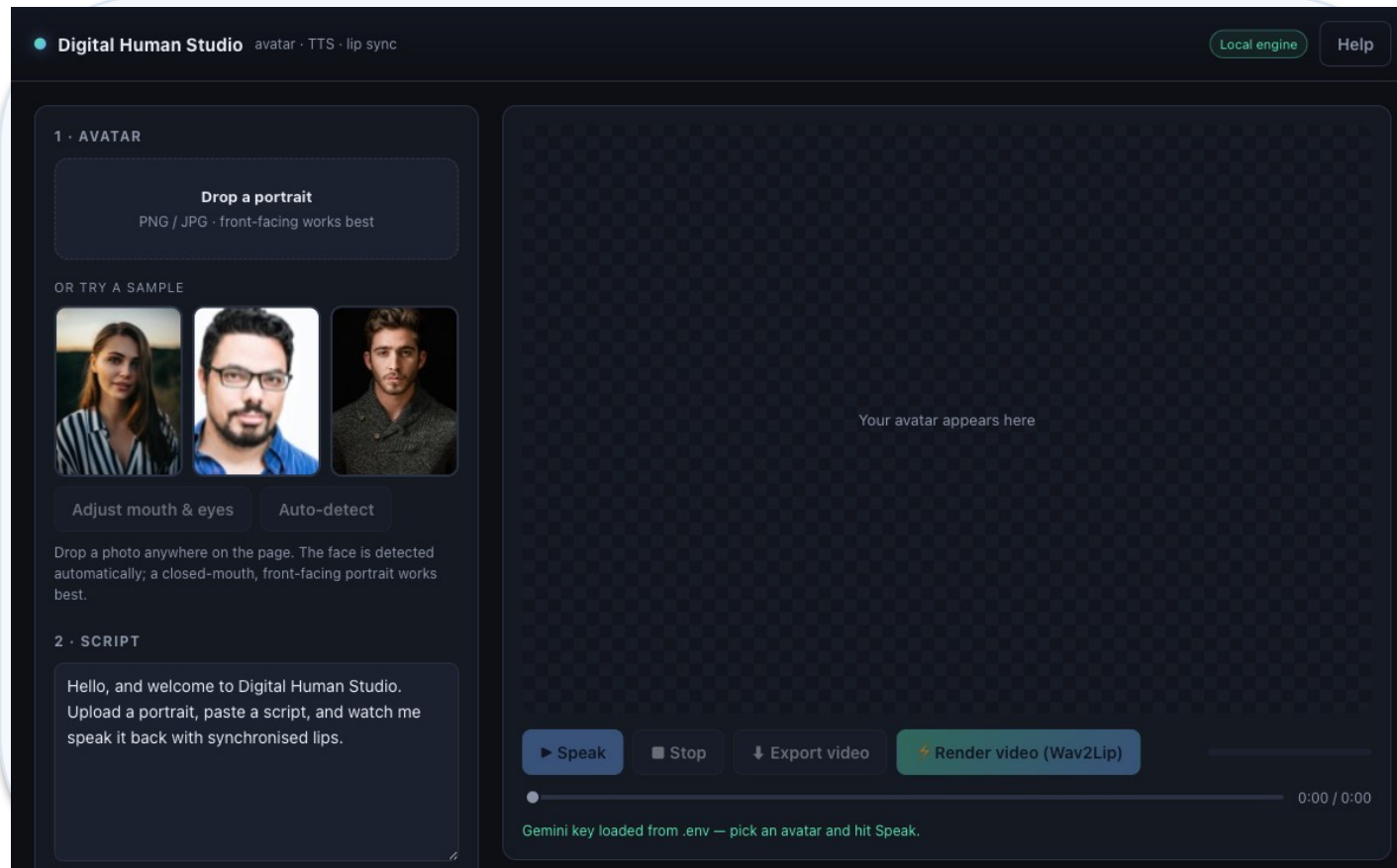
Build one observable behavior at a time.

Lab 6 - Digital Human Studio (<http://localhost:8137>)



Lab 6 - Digital Human Studio (<http://localhost:8137>): face detected, script loaded, and the avatar speaking in the live preview.

Lab 6 - the controls that matter



Lab 6 - the controls that matter: Draft with Ollama (gemma4), the voice engine, and the LIP SYNC renderer picker.

Lab 6: Prove, Evaluate, Improve

OK

Both clips render from the identical script and portrait.

OK

You can state, in one sentence each, when you would choose each engine - backed by what you saw.

OK

Run one edge case and record the change made after evaluation.

EVIDENCE STACK

01

Input

Exact test payload

02

Trace

Node execution path

03

Output

Generated result

04

Decision

Rubric and revision

PASS

Lab 6: Diagnose Before Rebuilding

SYMPTOM	LIKELY CAUSE	TARGETED FIX
'Could not load that sample'	The studio was opened from file://.	Always launch via start.command / start.bat - the browser blocks fetch() on file URLs.
MuseTalk render fails	Weights not downloaded, or no GPU/MPS available.	Re-run ./setup.sh --musetalk and read the service log.
HeyGen render rejected	Missing API key or exhausted credits.	Add the key in the studio settings and check the credit balance.

Change one variable, rerun the same test, compare the trace.

Lab 7: Avatar News Video with HeyGen (GG News Studio)

7

YOU WILL BUILD

Facts in, broadcast out: gemma4 writes spoken copy, HeyGen renders a presenter reading it, and the page polls until the video plays.

DELIVERABLE

Your own news broadcast, played to the class, plus quality-review notes on the script and the render.

T

TIMEBOX

45 minutes

M

MODE

Demo, guided build, independent test

E

EVIDENCE

Execution trace plus learner artifact

Success is demonstrated, not assumed.

Lab 7: Four Ideas That Make It Work

1

Two texts

You type FACTS; gemma4 writes SPOKEN COPY. The renderer speaks every character it is given - feed it raw facts and the avatar reads out a list.

2

Generate + poll

Rendering is far too slow for one HTTP request: /heygen-generate returns a video ID at once, and the page polls /heygen-status.

3

Spoken-copy rules

No bullets, no URLs, no markdown - a stray asterisk becomes an audible 'asterisk'.

4

Honest progress

HeyGen reports only processing or completed, so the bar estimates elapsed time and only reaches 100% when the render truly finishes.

Connect the four ideas before touching the workflow canvas.

Lab 7: Build Sequence

1

Import lab7/heygen-news-avatar-flow.json and set it Active.

2

Add the HeyGen API credential, and confirm the Ollama local credential on the script node.

3

Serve the site from lab7/website and open the GG News Studio.

4

Write YOUR OWN news item - a story from your industry, your company or today's headlines - and generate; watch gemma4's spoken script appear in the teleprompter.

5

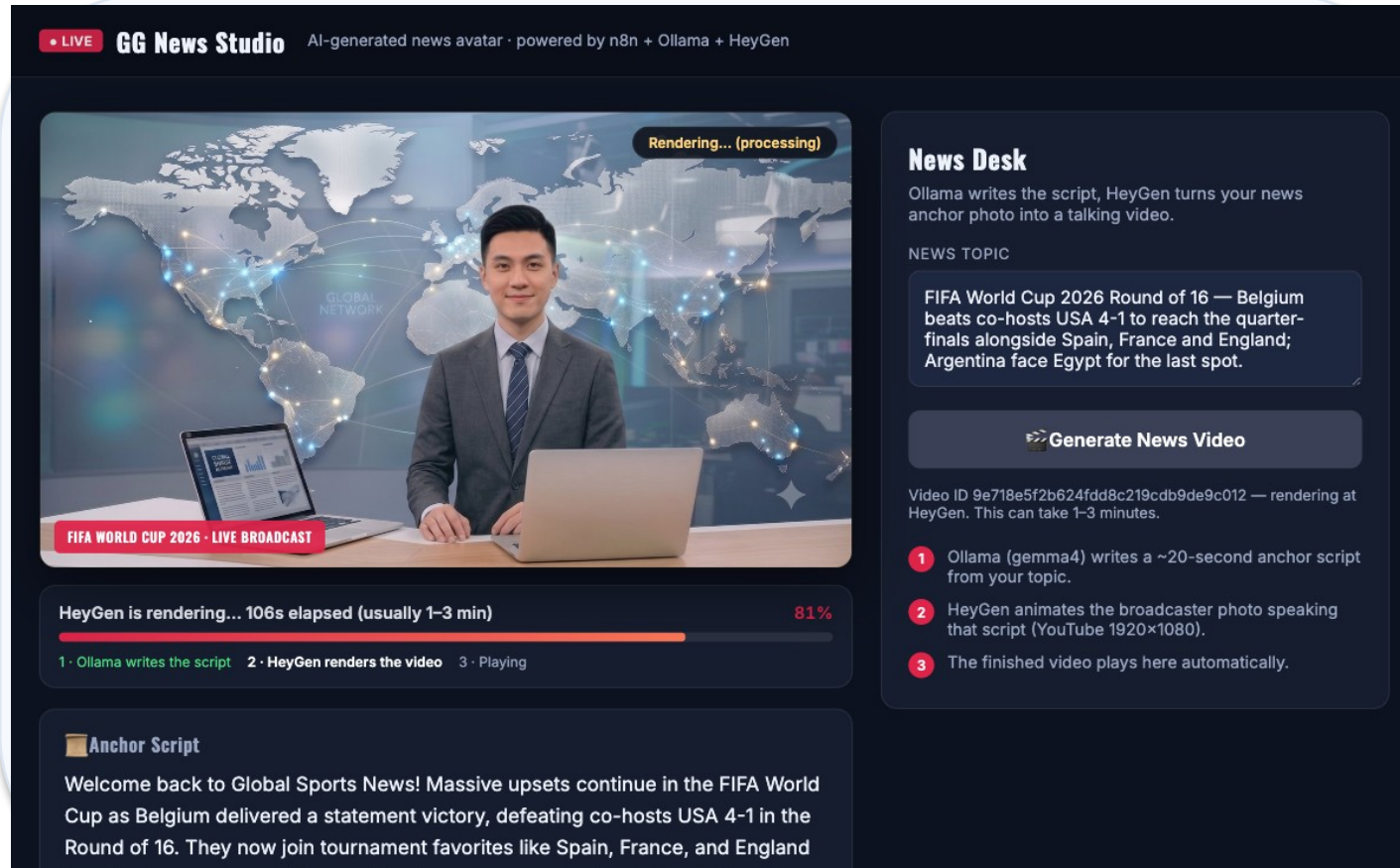
Watch the page poll /heygen-status until the video is completed, then play it. Read the script aloud: does it SOUND spoken?

6

SHARE IT: play your broadcast to the class, and note one piece of feedback you would act on.

Build one observable behavior at a time.

Lab 7 - the GG News Studio



Lab 7 - the GG News Studio: gemma4's script in the teleprompter and an honest render progress bar.

Lab 7: Prove, Evaluate, Improve

OK

The script reads as speech - no bullets, URLs or markdown.

OK

The page polls and eventually plays the finished video.

OK

YOUR OWN news broadcast is generated and played to the class.

OK

Run one edge case and record the change made after evaluation.

EVIDENCE STACK

01

Input

Exact test payload

02

Trace

Node execution path

03

Output

Generated result

04

Decision

Rubric and revision

PASS

Lab 7: Diagnose Before Rebuilding

SYMPTOM	LIKELY CAUSE	TARGETED FIX
The request times out	The flow tried to render inside a single HTTP request.	Keep the split shape: generate returns the ID immediately; the page polls status.
The avatar says 'asterisk'	Markdown leaked into the script.	Tighten the system prompt: spoken copy only, no formatting characters.
Status never reaches completed	Wrong video ID, or HeyGen credits ran out.	Read the status execution in n8n and check the HeyGen account.

Change one variable, rerun the same test, compare the trace.

Lab 7-os: Open-Source News Avatar - Free and Local

7-os

YOU WILL BUILD

The same news video with zero cloud and zero credits: TTS, Wav2Lip and ffmpeg on your own machine, driven by n8n.

DELIVERABLE

A locally rendered news video and a written HeyGen-versus-local comparison.

T

TIMEBOX

45 minutes

M

MODE

Demo, guided build, independent test

E

EVIDENCE

Execution trace plus learner artifact

Success is demonstrated, not assumed.

Lab 7-os: Four Ideas That Make It Work

1

Local render service

start.command / start.bat runs the pipeline - TTS to speech.wav, Wav2Lip lip-sync, ffmpeg to 1920x1080 - at localhost:8099/render.

2

n8n calls OUT

n8n reaches your machine at host.docker.internal - the opposite direction from Lab 4's tools, so no tunnel is needed.

3

One-response shape

A ~16 s local render is fast enough to answer in a single response - no video ID, no polling.

4

The trade

Wav2Lip's mouth is generated at 96x96 and soft at 1080p - but the render is free, private and fast.

Connect the four ideas before touching the workflow canvas.

Lab 7-os: Build Sequence

1

Start the render service FIRST: start.command (macOS) or start.bat (Windows), and leave the window open.

2

Import lab7-opensource/os-news-avatar-flow.json and set it Active.

3

Open the lab website and submit the same facts you used in Lab 7.

4

Watch the execution: n8n calls the render service at host.docker.internal:8099/render and waits for the finished file.

5

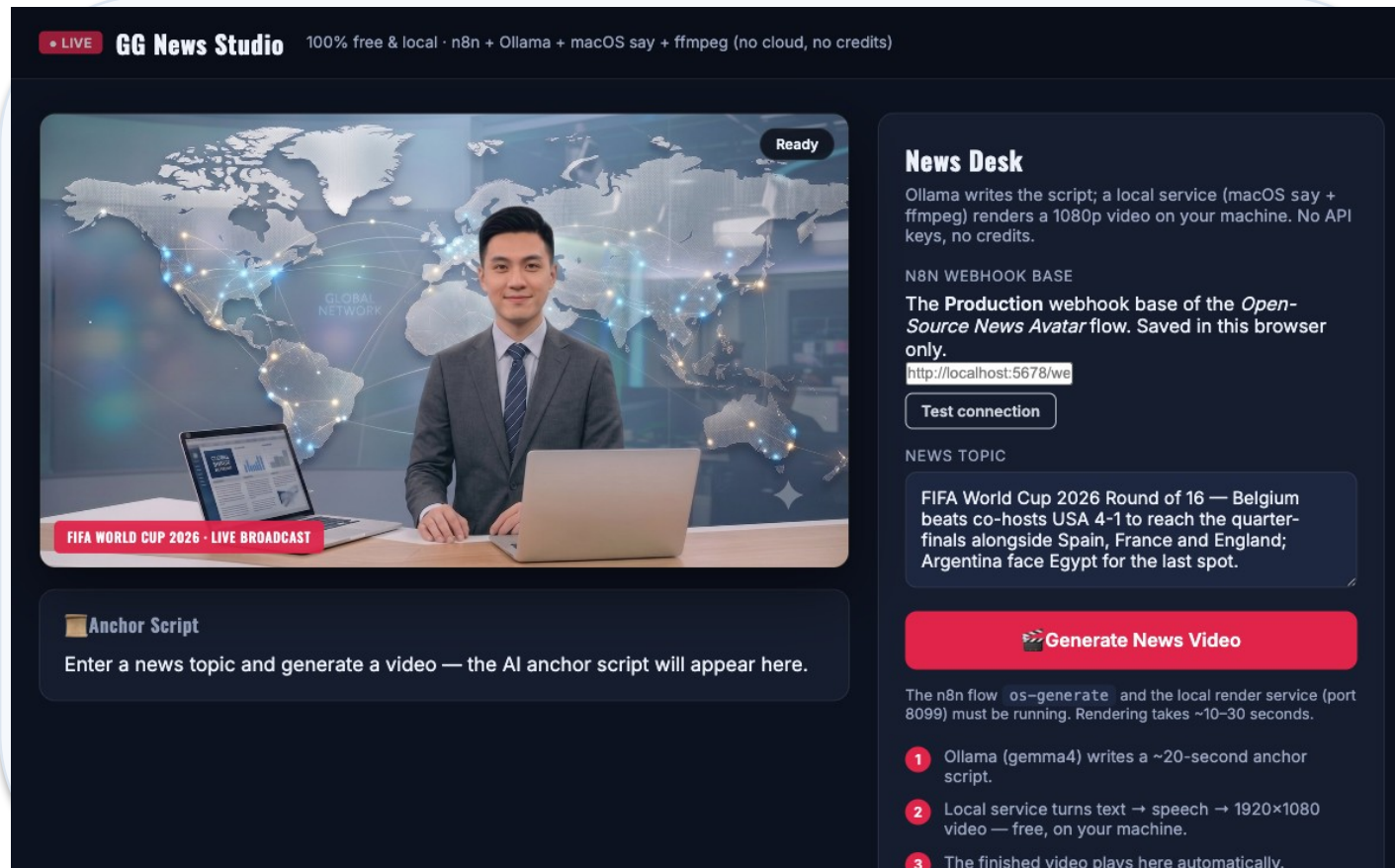
Play the finished MP4 in the page - produced entirely on your machine.

6

Compare it against the HeyGen clip from Lab 7: quality, speed, privacy, cost.

Build one observable behavior at a time.

Lab 7-os - the open-source News Studio



Lab 7-os - the open-source News Studio: 100% free and local - n8n + Ollama + TTS + ffmpeg, no cloud, no credits.

Lab 7-os: Prove, Evaluate, Improve

OK

A finished MP4 plays in the page, rendered locally.

OK

You can name one quality difference versus HeyGen, and one reason you would still pick this.

OK

Run one edge case and record the change made after evaluation.

EVIDENCE STACK

01

Input

Exact test payload

02

Trace

Node execution path

03

Output

Generated result

04

Decision

Rubric and revision

PASS

Lab 7-os: Diagnose Before Rebuilding

SYMPTOM	LIKELY CAUSE	TARGETED FIX
n8n cannot reach the render service	It called localhost instead of the host machine.	Use host.docker.internal:8099 - n8n runs inside Docker.
Nothing renders	The render service was never started.	Run start.command / start.bat first and keep that window open.
Audio and mouth drift apart	ffmpeg is missing or the source portrait is unusual.	Install ffmpeg and start from the provided portrait.

Change one variable, rerun the same test, compare the trace.

Lab 8: Interactive Avatar Brain (Aria, In-Browser)

8

YOU WILL BUILD

A talking avatar you can interrupt: speech in, an AI Agent reply from local gemma4, and the mouth drawn live in the browser.

DELIVERABLE

A conversation with Aria, the latency-panel screenshot, and one prompt improvement you made and re-tested.

T

TIMEBOX

45 minutes

M

MODE

Demo, guided build, independent test

E

EVIDENCE

Execution trace plus learner artifact

Success is demonstrated, not assumed.

Lab 8: Four Ideas That Make It Work

1

AI Agent + Ollama

The same two nodes as Lab 1 - AI Agent plus local gemma4 - now behind a webhook instead of a chat window.

2

Stateless by design

The browser sends the transcript with every turn, so the flow needs no memory node - the system prompt carries the conversation.

3

Spoken register

One or two spoken sentences, no markdown, no URLs - Aria's prompt enforces it, and a cleanup node strips anything that slips through.

4

Latency is the feature

An interactive avatar lives or dies on response time. The page prints the measured reply time - watch it, do not guess it.

Connect the four ideas before touching the workflow canvas.

Lab 8: Build Sequence

1

Import lab8/avatar-chat-flow.json and set it Active. The shape: Webhook -> Build Prompt -> AI Agent (Ollama gemma4) -> Make It Speakable -> Respond.

2

Open the AI Agent node and read Aria's system prompt: the voice rules, the conversation rules, and the facts she may state.

3

Open Make It Speakable and see what it strips before the avatar speaks: markdown, URLs, and a thinking model's private reasoning.

4

Serve the lab8 website and talk to Aria.

5

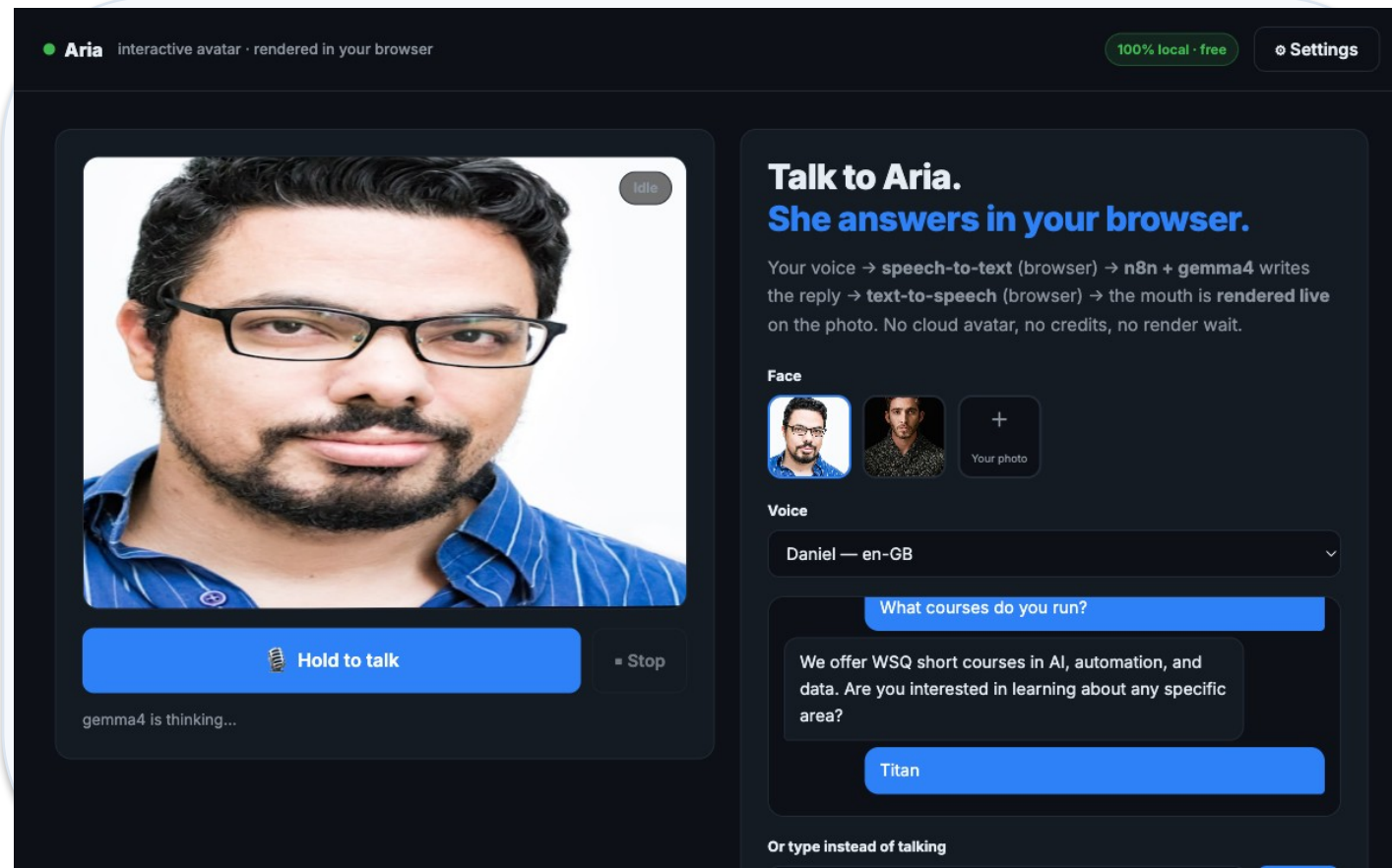
Watch the timing panel the page prints after each reply, and screenshot it - that is your evidence.

6

Tighten one prompt rule (for example: always end with one short follow-up question), re-test, and note what changed.

Build one observable behavior at a time.

Lab 8 - Aria, rendered in the browser



Lab 8 - Aria, rendered in the browser: speech in, gemma4 reply, and the mouth drawn live. The latency is printed, not claimed.

Lab 8: Prove, Evaluate, Improve

OK

Aria answers helpfully by voice - never 'I did not catch that' when the message arrived.

OK

Replies are one or two spoken sentences - no markdown, no URLs.

OK

The latency-panel screenshot is captured.

OK

Run one edge case and record the change made after evaluation.

EVIDENCE STACK

01

Input

Exact test payload

02

Trace

Node execution path

03

Output

Generated result

04

Decision

Rubric and revision

PASS

Lab 8: Diagnose Before Rebuilding

SYMPTOM	LIKELY CAUSE	TARGETED FIX
Aria gives an empty or 'hiccup' reply	The model returned nothing - often a thinking model spending its budget on private reasoning.	Open the execution and read the AI Agent output; the cleanup node's think-stripping must stay in place.
The first reply takes forever	The local model is cold-loading into memory.	Expected once per session - later replies are much faster. Close heavy apps if it persists.
Replies read like essays	The system prompt's voice rules were weakened.	Restore the rules: one or two spoken sentences, no markdown, no URLs.

Change one variable, rerun the same test, compare the trace.

Lab 9: Interactive Avatar Session (Nova, HeyGen LiveAvatar)

9

YOU WILL BUILD

A cloud interactive avatar embedded in the page through a short-lived session URL minted by n8n - the API key never reaches the browser.

DELIVERABLE

A working Nova session plus the two-column scorecard comparing her against the browser-rendered avatar.

T

TIMEBOX

45 minutes

M

MODE

Demo, guided build, independent test

E

EVIDENCE

Execution trace plus learner artifact

Success is demonstrated, not assumed.

Lab 9: Four Ideas That Make It Work

1

Session minting

n8n calls LiveAvatar with the API key server-side and returns only a short-lived embed URL to the page.

2

Cloud quality

Nova looks far better than Aria - photoreal, fluid - and costs credits for every session.

3

The comparison

Aria: free, private, ~2 s, drawn mouth. Nova: photoreal, cloud, credits. Both are defensible - be able to say when.

4

The same key instinct

Lab 4's signed URL, Lab 9's embed URL, Lab 10's proxy URL - the browser only ever receives short-lived, single-purpose tokens.

Connect the four ideas before touching the workflow canvas.

Lab 9: Build Sequence

1

Import lab9/liveavatar-session-flow.json and set it Active.

2

Add the LiveAvatar API credential in n8n.

3

Serve the lab9 website and start a session with Nova.

4

Hold a short conversation and note the quality and the latency.

5

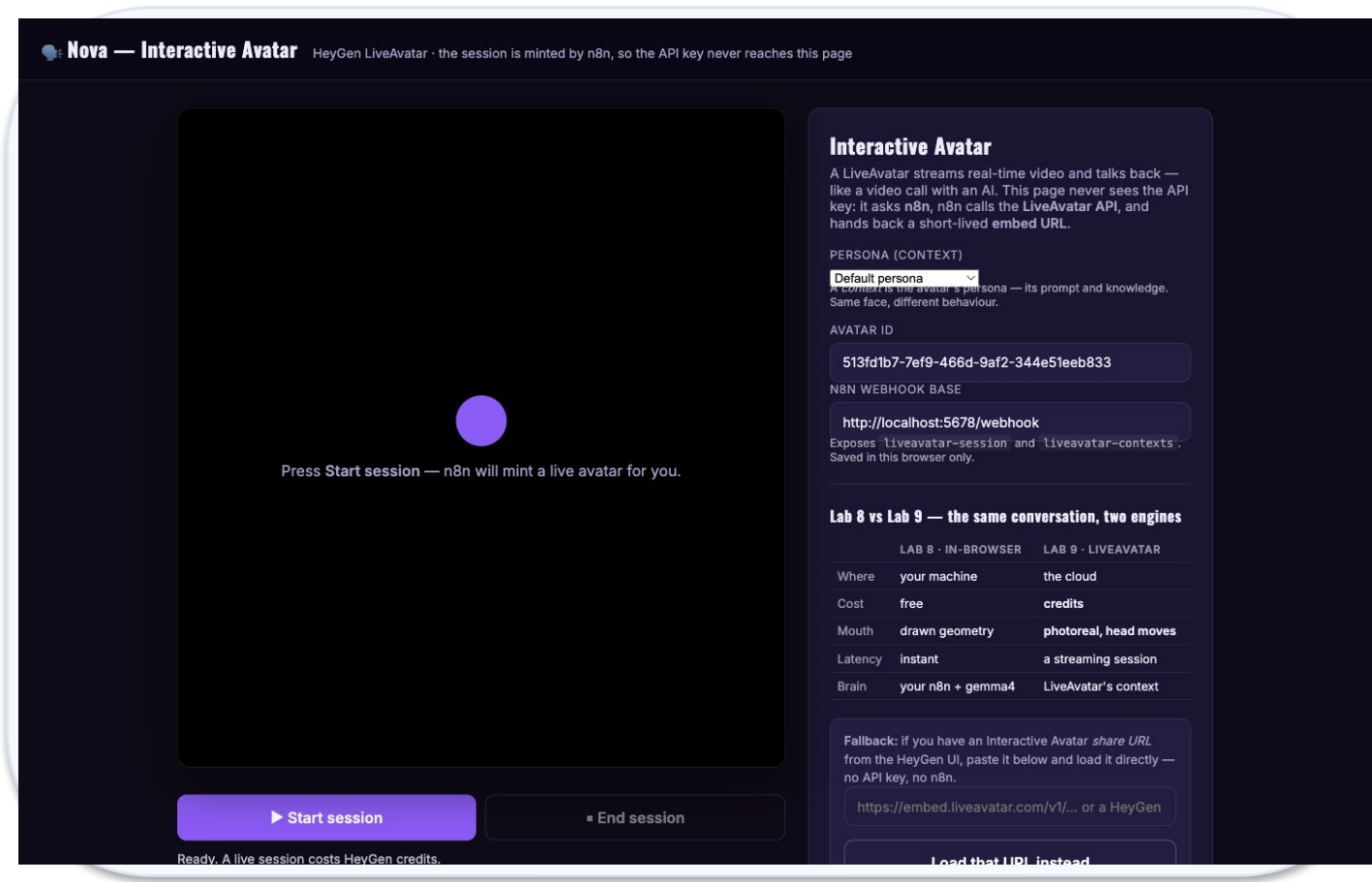
Write down one thing Nova does better than Aria, and one thing Aria does better than Nova.

6

Open the execution and find the session URL - confirm no API key ever reached the page.

Build one observable behavior at a time.

Lab 9 - the LiveAvatar embed



Lab 9 - the LiveAvatar embed: n8n mints a short-lived session URL, so the API key never reaches the browser.

Lab 9: Prove, Evaluate, Improve

OK

Nova loads and holds a conversation.

OK

The Aria-versus-Nova scorecard has one honest entry in each column.

OK

Run one edge case and record the change made after evaluation.

EVIDENCE STACK

01

Input

Exact test payload

02

Trace

Node execution path

03

Output

Generated result

04

Decision

Rubric and revision

PASS

Lab 9: Diagnose Before Rebuilding

SYMPTOM	LIKELY CAUSE	TARGETED FIX
The session fails to start	The LiveAvatar credential is missing or invalid.	Fix the credential in n8n and read the execution error.
The embed loads, then dies	The short-lived session URL expired.	Mint a fresh session - expiring quickly is the point of these URLs.
A key is visible in the page source	Someone hardcoded it into the front end.	Remove it - the page must only ever receive the embed URL.

Change one variable, rerun the same test, compare the trace.

Lab 10: AI Video Generation with Gemini Veo 3 (Veo Studio)

10

YOU WILL BUILD

One sentence in, an 8-second cinematic clip with sound out: gemma4 writes the shot prompt, Veo 3.1 renders it, n8n proxies the file.

DELIVERABLE

Your own Veo clip presented to the class, the shot prompt that produced it, and a note on what changed between two prompt versions.

T

TIMEBOX

60 minutes

M

MODE

Demo, guided build, independent test

E

EVIDENCE

Execution trace plus learner artifact

Success is demonstrated, not assumed.

Lab 10: Four Ideas That Make It Work

1

Shot prompt

gemma4 turns your idea into camera, lighting and motion language - a shot, not a summary.

2

Generate, poll, fetch

The same long-running shape as Lab 7 - plus a third step to fetch the finished file.

3

The /veo-file proxy

Google's download URL demands your API key, so n8n hands the page a URL pointing back at ITSELF, fetches the MP4 server-side, and streams it through.

4

Prompt iteration

Change the shot wording, regenerate, and record what changed - that note is the deliverable.

Connect the four ideas before touching the workflow canvas.

Lab 10: Build Sequence

1

Import lab10/veo3-video-flow.json and set it Active.

2

Add the Gemini API credential, and confirm the Ollama local credential.

3

Serve the lab10 website and open the Veo Studio.

4

Type YOUR OWN idea - one sentence, any subject you like - and generate; then read the shot prompt gemma4 actually wrote.

5

Wait for the poll to complete and play the clip IN the page, through the /veo-file proxy. Refine the prompt once and regenerate.

6

SHARE IT: present your best clip to the class, and say what changed between your two prompt versions.

Build one observable behavior at a time.



Lab 10: Prove, Evaluate, Improve

OK

The shot prompt names camera, lighting and motion - not a summary.

OK

The clip plays in the page.

OK

YOUR OWN clip is presented to the class, with the before/after prompt note.

OK

You can explain why the page never sees the Gemini key.

OK

Run one edge case and record the change made after evaluation.

EVIDENCE STACK

01

Input

Exact test payload

02

Trace

Node execution path

03

Output

Generated result

04

Decision

Rubric and revision

PASS

Lab 10: Diagnose Before Rebuilding

SYMPTOM	LIKELY CAUSE	TARGETED FIX
The clip will not play in the page	The page tried Google's URL directly and was refused.	Play it through the /veo-file proxy - n8n adds the key server-side.
Generation fails immediately	API quota, or a malformed prompt payload.	Run a short test prompt and read the provider response in the execution.
The clip ignores the idea	The shot prompt drifted from the subject.	Tighten the subject, camera and lighting wording, then regenerate.

Change one variable, rerun the same test, compare the trace.

Four Businesses, Four Agents

COOK & BAKE ACADEMY

Lab 3

Website course advisor, grounded ONLY in the brochures - honest about what it does not know.

GG HAIR SALON

Lab 4

Voice receptionist who checks the REAL Google Calendar and books into it.

MEDIREFILL

Lab 5

Refill voice FAQ that never gives medical advice - refusal + pharmacist callback.

GG NEWS STUDIO

Labs 7 / 7-os

Presenter video from the day's facts - and a cloud vs open-source recommendation.

Keep your workflow exports, traces and transcripts - together they are your portfolio from this course.

Evidence Rubric

1	Problem definition	User, trigger, output and non-goals	
2	Workflow correctness	Expected paths proven by execution	
3	AI quality	Prompts, retrieval and media evaluated	
4	Guardrails	Secrets, claims and publishing controlled	
5	Documentation	Another operator can run and recover	

Score the evidence, explain one improvement, then rerun the test.

Course Feedback

Before you leave - two minutes that shape the next run of this course.

1

Share your feedback

Fill in the short feedback form from the trainer

2

Rate the labs

Tell us which builds worked and which need more time

3

Collect your certificate

Your certificate of completion is emailed after the course

Thank you for taking part. Keep your workflow exports - they are your portfolio.

Thank You

A polished output is only the beginning.

Professional agentic systems are observable, testable, guarded, recoverable, and documented.

TRACE

TEST

REVIEW

RECOVER

DOCUMENT

www.tertiarycourses.com.sg